

```
// ===== File: com/ztransfer/AppLocale.kt =====
package com.ztransfer
import android.content.Context
import android.content.res.Configuration
import java.util.Locale
/**
 * 应用内语言切换。偏好存 SharedPreferences("ztransfer") 的 "app_language",
 * 值为 BCP-47 标签("en" / "zh-Hans" / "zh-Hant")或 [SYSTEM] (跟随系统)。
 * 所有取用户可见文案的 Context (Activity 基座、ViewModel 错误文案、服务通知)
 * 都先经 [wrap] 包装, 保证界面、错误信息与通知语言一致。
 */
object AppLocale {
    const val PREF_KEY = "app_language"
    const val SYSTEM = "system"
    fun wrap(base: Context): Context {
        val tag = base.getSharedPreferences("ztransfer", Context.MODE_PRIVATE)
            .getString(PREF_KEY, SYSTEM) ?: SYSTEM
        if (tag == SYSTEM) return base
        // 不动 Locale.setDefault: 它是进程级全局且切回"跟随系统"后无法恢复(粘滞),
        // 而本应用没有任何读 Locale.getDefault() 的代码——资源本地化靠配置上下文即可。
        val locale = Locale.forLanguageTag(tag)
        val config = Configuration(base.resources.configuration)
        config.setLocale(locale)
        return base.createConfigurationContext(config)
    }
}
// ===== File: com/ztransfer/frame/PhotoFrameExporter.kt =====
package com.ztransfer.frame
import android.content.ContentResolver
import android.content.Context
import android.graphics.Bitmap
import android.graphics.BitmapFactory
import android.graphics.Canvas
import android.graphics.Color
import android.graphics.ColorSpace
import android.graphics.ImageDecoder
import android.graphics.LinearGradient
import android.graphics.Matrix
import android.graphics.Paint
import android.graphics.Path
import android.graphics.Rect
import android.graphics.RectF
import android.graphics.Shader
import android.graphics.Typeface
import android.net.Uri
import android.os.Build
import android.provider.DocumentsContract
import android.provider.OpenableColumns
import androidx.exifinterface.media.ExifInterface
import androidx.core.content.res.ResourcesCompat
```

```
import com.ztransfer.R
import java.io.BufferedInputStream
import java.io.BufferedOutputStream
import java.io.File
import java.security.MessageDigest
import java.util.Locale
import java.util.UUID
import kotlin.math.min
import kotlin.math.pow
import kotlin.math.roundToInt
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.currentCoroutineContext
import kotlinx.coroutines.ensureActive
/** 未完成的边框派生文件；仅在完整写入后改成正式名称，App 下次启动会清理遗留项。 */
internal const val PHOTO_FRAME_PART_PREFIX = ".nkframe_"
internal const val PHOTO_FRAME_OUTPUT_DIRECTORY = "ZTframes"
private val PHOTO_FRAME_SESSION_PREFIX =
    "$PHOTO_FRAME_PART_PREFIX${UUID.randomUUID().toString().take(8)}_"
private const val PLAQUE_BAND_TO_WIDTH = 0.12f
/** 设置页可选的成片样式。名称是持久化键，不要随意改名。 */
enum class PhotoFramePreset(internal val fileSuffix: String) {
    MIST("mist"),
    CINEMA("dark"),
    MINIMAL("clean"),
    FROSTED("glass"),
    PLAQUE("plaque"),
}
/** 自定义水印选项。枚举名称会直接持久化，新增档位可以，已有名称不要修改。 */
enum class PhotoFrameWatermarkFont { SIGNATURE, ELEGANT, CALLIGRAPHY, SIMPLE, BOLD }
enum class PhotoFrameWatermarkContent { TEXT, IMAGE }
enum class PhotoFrameWatermarkPosition {
    AUTO,
    LEFT,
    CENTER,
    RIGHT,
    PHOTO_TOP_LEFT,
    PHOTO_TOP_CENTER,
    PHOTO_TOP_RIGHT,
    PHOTO_CENTER,
    PHOTO_BOTTOM_LEFT,
    PHOTO_BOTTOM_CENTER,
    PHOTO_BOTTOM_RIGHT,
}
enum class PhotoFrameWatermarkEffect { AUTO, NONE, SHADOW, OUTLINE }
enum class PhotoFrameWatermarkColor {
    ADAPTIVE,
    WHITE,
    BLACK,
    GOLD,
    MIST_BLUE,
```

```

    ROSE_GOLD,
}
data class PhotoFrameWatermark(
    val enabled: Boolean = true,
    val content: PhotoFrameWatermarkContent = PhotoFrameWatermarkContent.TEXT,
    val text: String = DEFAULT_PHOTO_FRAME_WATERMARK_TEXT,
    val imageHash: String? = null,
    val font: PhotoFrameWatermarkFont = PhotoFrameWatermarkFont.ELEGANT,
    val sizePercent: Int = DEFAULT_PHOTO_FRAME_WATERMARK_SIZE_PERCENT,
    val position: PhotoFrameWatermarkPosition = PhotoFrameWatermarkPosition.AUTO,
    val color: PhotoFrameWatermarkColor = PhotoFrameWatermarkColor.ADAPTIVE,
    val opacityPercent: Int = DEFAULT_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT,
    val effect: PhotoFrameWatermarkEffect = PhotoFrameWatermarkEffect.AUTO,
) {
    val displayText: String
        get() = limitPhotoFrameWatermarkText(text.trim())
            .ifEmpty { DEFAULT_PHOTO_FRAME_WATERMARK_TEXT }
}
internal const val DEFAULT_PHOTO_FRAME_WATERMARK_TEXT = "ZTransfer"
internal const val MAX_PHOTO_FRAME_WATERMARK_LENGTH = 24
internal const val MIN_PHOTO_FRAME_WATERMARK_SIZE_PERCENT = 1
internal const val MAX_PHOTO_FRAME_WATERMARK_SIZE_PERCENT = 200
internal const val DEFAULT_PHOTO_FRAME_WATERMARK_SIZE_PERCENT = 75
internal const val MIN_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT = 1
internal const val MAX_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT = 100
internal const val DEFAULT_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT = 72
internal const val PHOTO_FRAME_WATERMARK_IMAGE_DIRECTORY = "photo-frame-watermarks"
private val PHOTO_FRAME_WATERMARK_IMAGE_HASH = Regex("[0-9a-f]{64}")
private val PHOTO_FRAME_WATERMARK_LINE_BREAKS = Regex("[\\r\\n\\t]+")
private val PHOTO_FRAME_SOURCE_EXTENSIONS = setOf("jpg", "jpeg", "png")
internal fun isSupportedPhotoFrameSourceExtension(extension: String): Boolean =
    extension.removePrefix(".").lowercase(Locale.ROOT) in PHOTO_FRAME_SOURCE_EXTENSIONS
/** 保持单行并按 Unicode code point 截断，避免粘贴控制符或切断 emoji 代理对。 */
internal fun limitPhotoFrameWatermarkText(value: String): String {
    val singleLine = value
        .replace(PHOTO_FRAME_WATERMARK_LINE_BREAKS, " ")
        .filterNot(Char::isISOControl)
    val count = singleLine.codePointCount(0, singleLine.length)
    if (count <= MAX_PHOTO_FRAME_WATERMARK_LENGTH) return singleLine
    return singleLine.substring(
        0,
        singleLine.offsetByCodePoints(0, MAX_PHOTO_FRAME_WATERMARK_LENGTH),
    )
}
internal fun normalizePhotoFrameWatermarkSizePercent(value: Int): Int =
    value.coerceIn(
        MIN_PHOTO_FRAME_WATERMARK_SIZE_PERCENT,
        MAX_PHOTO_FRAME_WATERMARK_SIZE_PERCENT,
    )
internal fun normalizePhotoFrameWatermarkOpacityPercent(value: Int): Int =

```

```

        value.coerceIn(
            MIN_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT,
            MAX_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT,
        )
    /** 百分比连续变化，同时精确经过旧小/中/大三个尺寸锚点，升级后像素尺寸不跳变。 */
    internal fun photoFrameWatermarkTextSizeFraction(sizePercent: Int): Float =
        piecewiseWatermarkSizeFraction(
            sizePercent = sizePercent,
            smallPercent = 58,
            smallFraction = 0.0105f,
            mediumPercent = 75,
            mediumFraction = 0.0135f,
            largeFraction = 0.018f,
        )
    internal fun photoFrameWatermarkImageSizeFraction(sizePercent: Int): Float =
        piecewiseWatermarkSizeFraction(
            sizePercent = sizePercent,
            smallPercent = 47,
            smallFraction = 0.035f,
            mediumPercent = 69,
            mediumFraction = 0.052f,
            largeFraction = 0.075f,
        )
    private fun piecewiseWatermarkSizeFraction(
        sizePercent: Int,
        smallPercent: Int,
        smallFraction: Float,
        mediumPercent: Int,
        mediumFraction: Float,
        largeFraction: Float,
    ): Float {
        val percent = normalizePhotoFrameWatermarkSizePercent(sizePercent)
        return when {
            percent <= smallPercent -> smallFraction * percent / smallPercent
            percent <= mediumPercent -> {
                val progress = (percent - smallPercent).toFloat() / (mediumPercent - smallPercent)
                smallFraction + (mediumFraction - smallFraction) * progress
            }
            percent <= 100 -> {
                val progress = (percent - mediumPercent).toFloat() / (100 - mediumPercent)
                mediumFraction + (largeFraction - mediumFraction) * progress
            }
            else -> largeFraction * percent / 100f
        }
    }
    internal fun validPhotoFrameWatermarkImageHash(value: String?): String? =
        value?.lowercase(Locale.ROOT)?.takeIf(PHOTO_FRAME_WATERMARK_IMAGE_HASH::matches)
    internal fun photoFrameWatermarkImageFile(context: Context, imageHash: String): File {
        val safeHash = requireNotNull(validPhotoFrameWatermarkImageHash(imageHash)) {
            "Invalid watermark image hash"
        }
    }

```

```
}
    return File(File(context.filesDir, PHOTO_FRAME_WATERMARK_IMAGE_DIRECTORY), "$safeHash.image")
}
data class PhotoFrameExportResult(
    val displayName: String,
)
internal data class PhotoFrameDestination(
    val directoryUri: Uri,
    val occupiedNames: MutableSet<String>,
)
internal data class PhotoFrameLayout(
    val canvasWidth: Int,
    val canvasHeight: Int,
    val photoLeft: Float,
    val photoTop: Float,
    val photoRight: Float,
    val photoBottom: Float,
    val metadataTop: Float,
)
internal data class PhotoFrameMetadata(
    val make: String?,
    val model: String?,
    val aperture: String?,
    val shutter: String?,
    val iso: String?,
    val focalLength: String?,
    val dateTime: String? = null,
)
internal data class FrameTextVisualBounds(
    /** 相对基线的顶部坐标，通常为负数。 */
    val top: Float,
    /** 相对基线的底部坐标，通常为零或正数。 */
    val bottom: Float,
)
internal data class PhotoWatermarkTextBounds(
    val left: Float,
    val top: Float,
    val right: Float,
    val bottom: Float,
)
internal data class PhotoWatermarkPlacement(
    val originX: Float,
    val baseline: Float,
)
/**
 * 边框导出器：读取已传输原片，在原片外创建新画布并另存 JPG。
 *
 * 接收 Android 可稳定解码的 JPG/JPEG/PNG。各代 NEF 无法可靠解码，强行支持会在不同
 * 手机上产生黑图或方向错误；RAW+JPEG 拍摄时应选择 JPG 成员生成分享图。
 */
```

```

object PhotoFrameExporter {
    // 分享图保留 3200px 长边（常规 4:3 约 7.7MP），手机端观看和二次发布已有
    // 充足余量；相比 4096px 可把“解码原片 + 输出画布”的峰值内存降低约 39%，
    // 对 128/192MB heap 的旧机和定制系统更可靠。原片本身始终无损保留。
    private const val MAX_SOURCE_EDGE = 3200
    private const val JPEG_QUALITY = 95
    private const val COPY_BUFFER_BYTES = 256 * 1024
    private val EMPTY_METADATA =
        PhotoFrameMetadata(null, null, null, null, null, null)
    private val bundledTypefaceCache = mutableMapOf<PhotoFrameWatermarkFont, Typeface>()
    private val watermarkImageCache = linkedMapOf<String, Bitmap>()
    internal suspend fun export(
        context: Context,
        resolver: ContentResolver,
        destination: PhotoFrameDestination,
        sourceUri: Uri,
        sourceName: String,
        preset: PhotoFramePreset,
        watermark: PhotoFrameWatermark,
        borderEnabled: Boolean = true,
    ): Result<PhotoFrameExportResult> {
        return try {
            currentCoroutineContext().ensureActive()
            require(
                isSupportedPhotoFrameSourceExtension(sourceName.substringAfterLast('.', "")),
            ) {
                "Only JPG/JPEG/PNG supports borders or watermarks"
            }
            val renderedWatermark = watermark.forBorderMode(borderEnabled)
            val metadata = if (borderEnabled) readMetadata(resolver, sourceUri) else EMPTY_METADATA
            val bitmap = decodeForFraming(resolver, sourceUri)
                ?: error("Cannot decode transferred original")
            val rendered = try {
                renderFrame(context, bitmap, metadata, preset, renderedWatermark, borderEnabled)
            } finally {
                bitmap.recycle()
            }
            val saved = try {
                currentCoroutineContext().ensureActive()
                saveRendered(
                    resolver = resolver,
                    destination = destination,
                    sourceName = sourceName,
                    preset = preset,
                    watermark = renderedWatermark,
                    borderEnabled = borderEnabled,
                    bitmap = rendered,
                )
            } finally {
                rendered.recycle()
            }
        }
    }
}

```

```

    }
    Result.success(saved)
} catch (cancelled: CancellationException) {
    throw cancelled
} catch (outOfMemory: OutOfMemoryError) {
    // 派生图允许优雅失败，原片已经安全落盘；资源均由上面的 finally 回收。
    Result.failure(outOfMemory)
} catch (error: Exception) {
    Result.failure(error)
}
}
}
/**
 * 在用户选择的保存目录下复用或创建固定成片目录，并只在首次准备时扫描一次已有名称。
 * 调用方按根目录缓存结果；边框线程单并发，后续名称由 [saveRendered] 增量登记。
 */
internal fun prepareDestination(
    resolver: ContentResolver,
    treeUri: Uri,
): PhotoFrameDestination {
    val treeId = DocumentsContract.getTreeDocumentId(treeUri)
    val rootUri = DocumentsContract.buildDocumentUriUsingTree(treeUri, treeId)
    val directoryUri = findFrameDirectory(resolver, treeUri, treeId)
    ?: DocumentsContract.createDocument(
        resolver,
        rootUri,
        DocumentsContract.Document.MIME_TYPE_DIR,
        PHOTO_FRAME_OUTPUT_DIRECTORY,
    )
    ?: error("Cannot create frame output directory")
    return PhotoFrameDestination(
        directoryUri = directoryUri,
        occupiedNames = listChildNames(resolver, treeUri, directoryUri).toMutableSet(),
    )
}
private fun findFrameDirectory(
    resolver: ContentResolver,
    treeUri: Uri,
    parentDocumentId: String,
): Uri? {
    val childrenUri = DocumentsContract.buildChildDocumentsUriUsingTree(
        treeUri,
        parentDocumentId,
    )
    return resolver.query(
        childrenUri,
        arrayOf(
            DocumentsContract.Document.COLUMN_DOCUMENT_ID,
            DocumentsContract.Document.COLUMN_DISPLAY_NAME,
            DocumentsContract.Document.COLUMN_MIME_TYPE,
        ),
    ),

```

```
        null,
        null,
        null,
    )?.use { cursor ->
        val idColumn =
            cursor.getColumnIndex/DocumentsContract.Document.COLUMN_DOCUMENT_ID)
        val nameColumn =
            cursor.getColumnIndex/DocumentsContract.Document.COLUMN_DISPLAY_NAME)
        val mimeTypeColumn =
            cursor.getColumnIndex/DocumentsContract.Document.COLUMN_MIME_TYPE)
        if (idColumn < 0 || nameColumn < 0 || mimeTypeColumn < 0) return@use null
        while (cursor.moveToNext()) {
            val isTarget =
                cursor.getString(nameColumn).equals(
                    PHOTO_FRAME_OUTPUT_DIRECTORY,
                    ignoreCase = true,
                ) &&
                cursor.getString(mimeTypeColumn) ==
                    DocumentsContract.Document.MIME_TYPE_DIR
            if (isTarget) {
                return@use DocumentsContract.buildDocumentUriUsingTree(
                    treeUri,
                    cursor.getString(idColumn),
                )
            }
        }
        null
    }
}

private fun listChildNames(
    resolver: ContentResolver,
    treeUri: Uri,
    directoryUri: Uri,
): Set<String> {
    val documentId = DocumentsContract.getDocumentId(directoryUri)
    val childrenUri =
        DocumentsContract.buildChildDocumentsUriUsingTree(treeUri, documentId)
    return resolver.query(
        childrenUri,
        arrayOf(
            DocumentsContract.Document.COLUMN_DOCUMENT_ID,
            DocumentsContract.Document.COLUMN_DISPLAY_NAME,
        ),
        null,
        null,
        null,
    )?.use { cursor ->
        val idColumn =
            cursor.getColumnIndex/DocumentsContract.Document.COLUMN_DOCUMENT_ID)
        val nameColumn =
```



```

        cursor.getColumnIndex/DocumentsContract.Document.COLUMN_DISPLAY_NAME)
    if (nameColumn < 0) return@use emptySet()
    buildSet {
        while (cursor.moveToNext()) {
            val name = cursor.getString(nameColumn) ?: continue
            if (name.startsWith(PHOTO_FRAME_PART_PREFIX)) {
                // 新目录把临时文件也放在内部；清理旧进程遗留项，但绝不碰本会话任务。
                if (
                    !isCurrentPhotoFrameTempName(name) &&
                    idColumn >= 0
                ) {
                    val childId = cursor.getString(idColumn) ?: continue
                    runCatching {
                        DocumentsContract.deleteDocument(
                            resolver,
                            DocumentsContract.buildDocumentUriUsingTree(
                                treeUri,
                                childId,
                            ),
                        )
                    }
                }
            } else {
                add(name)
            }
        }
    }
    } ?: emptySet()
}

private fun readMetadata(resolver: ContentResolver, uri: Uri): PhotoFrameMetadata {
    val descriptorResult = runCatching {
        resolver.openFileDescriptor(uri, "r")?.use { pfd ->
            metadataFrom(ExifInterface(pfd.fileDescriptor))
        }
    }.getOrNull()
    if (descriptorResult != null) return descriptorResult
    // 少数 DocumentsProvider 返回不可 seek 的文件描述符，但输入流仍可正常读取。
    return runCatching {
        resolver.openInputStream(uri)?.use { input ->
            BufferedInputStream(input).use { metadataFrom(ExifInterface(it)) }
        }
    }.getOrNull() ?: EMPTY_METADATA
}

private fun metadataFrom(exif: ExifInterface): PhotoFrameMetadata {
    val fNumber = exif.getAttributeDouble(ExifInterface.TAG_F_NUMBER, Double.NaN)
        .takeIf { it.isFinite() && it > 0.0 }
    ?: exif.getAttributeDouble(ExifInterface.TAG_APERTURE_VALUE, Double.NaN)
        .takeIf { it.isFinite() }
        ?.let { 2.0.pow(it / 2.0) }
    val exposureSeconds =

```

```

        exif.getAttributeDouble(ExifInterface.TAG_EXPOSURE_TIME, Double.NaN)
            .takeIf { it.isFinite() && it > 0.0 }
        ?: exif.getAttributeDouble(
            ExifInterface.TAG_SHUTTER_SPEED_VALUE,
            Double.NaN,
        ).takeIf { it.isFinite() }?.let { 2.0.pow(-it) }
    return PhotoFrameMetadata(
        make = exif.getAttribute(ExifInterface.TAG_MAKE),
        model = exif.getAttribute(ExifInterface.TAG_MODEL),
        aperture = fNumber?.let(::formatAperture),
        shutter = exposureSeconds?.let(::formatShutter),
        iso = normalizeIso(
            exif.getAttribute(ExifInterface.TAG_PHOTOGRAPHIC_SENSITIVITY),
        ),
        focalLength = exif.getAttributeDouble(
            ExifInterface.TAG_FOCAL_LENGTH,
            Double.NaN,
        ).takeIf { it.isFinite() && it > 0.0 }
            ?.let { String.format(Locale.US, "%.0fmm", it) },
        dateTime = sequenceOf(
            ExifInterface.TAG_DATETIME_ORIGINAL,
            ExifInterface.TAG_DATETIME_DIGITIZED,
            ExifInterface.TAG_DATETIME,
        ).mapNotNull(exif::getAttribute)
            .mapNotNull(::normalizeCaptureDateTime)
            .firstOrNull(),
    )
}

private fun decodeForFraming(resolver: ContentResolver, uri: Uri): Bitmap? {
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
        try {
            return ImageDecoder.decodeBitmap(
                ImageDecoder.createSource(resolver, uri),
            ) { decoder, info, _ ->
                val width = info.size.width
                val height = info.size.height
                val scale = min(1f, MAX_SOURCE_EDGE.toFloat() / maxOf(width, height))
                decoder.setTargetSize(
                    (width * scale).roundToInt().coerceAtLeast(1),
                    (height * scale).roundToInt().coerceAtLeast(1),
                )
                decoder.allocator = ImageDecoder.ALLOCATOR_SOFTWARE
                // 分享平台最终普遍以 sRGB 展示；显式色彩管理也能避免广色域 JPEG
                // 被解成 RGBA_F16，令解码位图内存意外翻倍。
                decoder.setTargetColorSpace(ColorSpace.get(ColorSpace.Named.SRGB))
            }
        } catch (_: Exception) {
            // 个别定制系统的 DocumentsProvider 可被 BitmapFactory 读取，却无法被
            // ImageDecoder.createSource 正常映射。回退到兼容路径；OOM 等 Error
            // 不在这里吞掉，避免内存不足时立刻再做一次同等规模解码。
        }
    }
}

```

```
    }
}
// Android 8, 或较新系统上 ImageDecoder 与定制 DocumentsProvider 不兼容时使用。
val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
resolver.openFileDescriptor(uri, "r")?.use {
    BitmapFactory.decodeFileDescriptor(it.fileDescriptor, null, bounds)
}
if (bounds.outWidth <= 0 || bounds.outHeight <= 0) return null
var sample = 1
while (maxOf(bounds.outWidth / sample, bounds.outHeight / sample) > MAX_SOURCE_EDGE) {
    sample *= 2
}
val decoded = resolver.openFileDescriptor(uri, "r")?.use {
    BitmapFactory.decodeFileDescriptor(
        it.fileDescriptor,
        null,
        BitmapFactory.Options().apply {
            inSampleSize = sample
            inPreferredConfig = Bitmap.Config.ARGB_8888
        },
    )
} ?: return null
val orientation = runCatching {
    resolver.openFileDescriptor(uri, "r")?.use {
        ExifInterface(it.fileDescriptor).getAttributeInt(
            ExifInterface.TAG_ORIENTATION,
            ExifInterface.ORIENTATION_NORMAL,
        )
    } ?: ExifInterface.ORIENTATION_NORMAL
}.getOrDefault(ExifInterface.ORIENTATION_NORMAL)
return try {
    applyOrientation(decoded, orientation)
} catch (error: Throwable) {
    decoded.recycle()
    throw error
}
}

private fun applyOrientation(source: Bitmap, orientation: Int): Bitmap {
    val matrix = Matrix()
    when (orientation) {
        ExifInterface.ORIENTATION_FLIP_HORIZONTAL -> matrix.setScale(-1f, 1f)
        ExifInterface.ORIENTATION_ROTATE_180 -> matrix.setRotate(180f)
        ExifInterface.ORIENTATION_FLIP_VERTICAL -> matrix.setScale(1f, -1f)
        ExifInterface.ORIENTATION_TRANSPOSE -> {
            matrix.setRotate(90f)
            matrix.postScale(-1f, 1f)
        }
        ExifInterface.ORIENTATION_ROTATE_90 -> matrix.setRotate(90f)
        ExifInterface.ORIENTATION_TRANSVERSE -> {
            matrix.setRotate(-90f)
        }
    }
}
```

```
        matrix.postScale(-1f, 1f)
    }
    ExifInterface.ROTATION_ROTATE_270 -> matrix.setRotate(-90f)
    else -> return source
}
val oriented = Bitmap.createBitmap(source, 0, 0, source.width, source.height, matrix, true)
if (oriented != source) source.recycle()
return oriented
}
internal fun renderPreview(
    context: Context,
    source: Bitmap,
    metadata: PhotoFrameMetadata,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark,
    borderEnabled: Boolean = true,
    longEdge: Int = 720,
): Bitmap = renderFrame(
    context,
    source,
    metadata,
    preset,
    watermark.forBorderMode(borderEnabled),
    borderEnabled,
    longEdge,
)
private fun renderFrame(
    context: Context,
    source: Bitmap,
    metadata: PhotoFrameMetadata,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark,
    borderEnabled: Boolean,
    longEdge: Int = 3200,
): Bitmap {
    require(longEdge > 0)
    if (!borderEnabled) {
        return renderWatermarkOnly(context, source, watermark, longEdge)
    }
    val layout = if (preset == PhotoFramePreset.PLAQUE) {
        calculatePlaqueFrameLayout(source.width, source.height, longEdge)
    } else {
        calculatePhotoFrameLayout(source.width, source.height, longEdge)
    }
    val output = Bitmap.createBitmap(
        layout.canvasWidth,
        layout.canvasHeight,
        Bitmap.Config.ARGB_8888,
    )
    try {
```

```
        val canvas = Canvas(output)
        if (preset == PhotoFramePreset.PLAQUE) {
            drawPlaqueFrame(context, canvas, source, layout, metadata, watermark)
            return output
        }
        drawBackdrop(canvas, source, preset)
        val photoRect = RectF(
            layout.photoLeft,
            layout.photoTop,
            layout.photoRight,
            layout.photoBottom,
        )
        // 照片和毛玻璃参数卡共用同一圆角尺度, 形成统一的卡片语言。
        val radius = photoFrameCornerRadius(layout)
        drawPhotoElevation(canvas, photoRect, radius, preset)
        val clip = Path().apply {
            addRoundRect(photoRect, radius, radius, Path.Direction.CW)
        }
        canvas.save()
        canvas.clipPath(clip)
        canvas.drawBitmap(
            source,
            null,
            photoRect,
            Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG or Paint.DITHER_FLAG),
        )
        drawPhotoWatermark(context, canvas, photoRect, preset, watermark)
        canvas.restore()
        Paint(Paint.ANTI_ALIAS_FLAG).apply {
            style = Paint.Style.STROKE
            strokeWidth = maxOf(1f, layout.canvasWidth * 0.0012f)
            color = if (preset != PhotoFramePreset.MINIMAL) {
                Color.argb(70, 255, 255, 255)
            } else {
                Color.argb(45, 20, 28, 35)
            }
            canvas.drawRoundRect(photoRect, radius, radius, this)
        }
        drawMetadata(
            context,
            canvas,
            layout,
            metadata,
            preset,
            watermark.withoutPhotoPlacement(),
        )
        return output
    } catch (error: Throwable) {
        output.recycle()
        throw error
    }
```

```

    }
}
/**
 * 两层柔影模拟照片浮放在衬纸上的高度：大而淡的环境影负责分离背景，小而稍深的
 * 落影给出方向。比单个浓重高斯阴影更自然，也不会形成廉价的黑色光晕。
 */
private fun drawPhotoElevation(
    canvas: Canvas,
    photoRect: RectF,
    radius: Float,
    preset: PhotoFramePreset,
) {
    val shortEdge = min(canvas.width, canvas.height).toFloat()
    val shadowStrength = when (preset) {
        PhotoFramePreset.CINEMA -> 1.15f
        PhotoFramePreset.MINIMAL -> 0.78f
        PhotoFramePreset.MIST, PhotoFramePreset.FROSTED -> 1f
        PhotoFramePreset.PLAQUE -> 0f
    }
    // ShadowLayer 在 3200px 画布上直接做两次软件模糊代价很高。阴影本身没有
    // 高频细节，先在 1/4 尺寸透明代理图渲染，再双线性放大，视觉一致而参与
    // 模糊的像素数约为原来的 1/16。
    val proxyScale = 0.25f
    val proxyWidth = (canvas.width * proxyScale).roundToInt().coerceAtLeast(1)
    val proxyHeight = (canvas.height * proxyScale).roundToInt().coerceAtLeast(1)
    val proxy = Bitmap.createBitmap(proxyWidth, proxyHeight, Bitmap.Config.ARGB_8888)
    try {
        val proxyCanvas = Canvas(proxy)
        val proxyRect = RectF(
            photoRect.left * proxyScale,
            photoRect.top * proxyScale,
            photoRect.right * proxyScale,
            photoRect.bottom * proxyScale,
        )
        val proxyRadius = radius * proxyScale
        Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.argb(18, 0, 0, 0)
            setShadowLayer(
                shortEdge * 0.020f * proxyScale,
                0f,
                shortEdge * 0.003f * proxyScale,
                Color.argb((48 * shadowStrength).roundToInt(), 8, 15, 21),
            )
            proxyCanvas.drawRoundRect(proxyRect, proxyRadius, proxyRadius, this)
            clearShadowLayer()
        }
        Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.argb(20, 0, 0, 0)
            setShadowLayer(
                shortEdge * 0.009f * proxyScale,

```

```

        0f,
        shortEdge * 0.009f * proxyScale,
        Color.argb((64 * shadowStrength).roundToInt(), 5, 11, 16),
    )
    proxyCanvas.drawRoundRect(proxyRect, proxyRadius, proxyRadius, this)
    clearShadowLayer()
}
canvas.drawBitmap(
    proxy,
    null,
    RectF(0f, 0f, canvas.width.toFloat(), canvas.height.toFloat()),
    Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG),
)
} finally {
    proxy.recycle()
}
}

private fun drawBackdrop(canvas: Canvas, source: Bitmap, preset: PhotoFramePreset) {
    when (preset) {
        PhotoFramePreset.MINIMAL -> {
            // 极轻的暖纸渐变比纯白更耐看，也能让白色照片边缘和阴影保持可见。
            Paint(Paint.ANTI_ALIAS_FLAG).apply {
                shader = LinearGradient(
                    0f,
                    0f,
                    0f,
                    canvas.height.toFloat(),
                    Color.rgb(250, 249, 247),
                    Color.rgb(239, 237, 232),
                    Shader.TileMode.CLAMP,
                )
                canvas.drawRect(
                    0f,
                    0f,
                    canvas.width.toFloat(),
                    canvas.height.toFloat(),
                    this,
                )
            }
        }
        PhotoFramePreset.MIST,
        PhotoFramePreset.CINEMA,
        PhotoFramePreset.FROSTED -> {
            // 先缩图，再做两轮可控盒式模糊，最后双线性放大。相比单纯把 72px 图硬拉大，
            // 渐变更连续、没有色块，同时不依赖 API 31 可用的 RenderEffect。
            val blurLongEdge = 192
            val blurWidth: Int
            val blurHeight: Int
            if (canvas.width >= canvas.height) {
                blurWidth = blurLongEdge
            }

```

```
        blurHeight =
            (blurLongEdge * canvas.height.toFloat() / canvas.width).roundToInt().coerceAtLeast(96)
    } else {
        blurHeight = blurLongEdge
        blurWidth =
            (blurLongEdge * canvas.width.toFloat() / canvas.height).roundToInt().coerceAtLeast(96)
    }
    val tiny = Bitmap.createBitmap(blurWidth, blurHeight, Bitmap.Config.ARGB_8888)
    try {
        Canvas(tiny).drawCenterCrop(
            source,
            RectF(0f, 0f, blurWidth.toFloat(), blurHeight.toFloat()),
        )
        blurBitmapInPlace(tiny, radius = 8, passes = 2)
        canvas.drawBitmap(
            tiny,
            null,
            RectF(0f, 0f, canvas.width.toFloat(), canvas.height.toFloat()),
            Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG),
        )
    } finally {
        tiny.recycle()
    }
    when (preset) {
        PhotoFramePreset.MIST -> {
            // 只轻提亮，不抹掉照片本身的主色；底部连续暗化，为白色品牌/参数提供
            // 稳定对比度。雪景会得到银灰质感，蓝天则保留克制的蓝色氛围。
            canvas.drawColor(Color.rgb(62, 238, 244, 248))
            Paint(Paint.ANTI_ALIAS_FLAG).apply {
                shader = LinearGradient(
                    0f,
                    canvas.height * 0.60f,
                    0f,
                    canvas.height.toFloat(),
                    Color.rgb(0, 3, 10, 15),
                    Color.rgb(178, 3, 10, 15),
                    Shader.TileMode.CLAMP,
                )
                canvas.drawRect(
                    0f,
                    canvas.height * 0.58f,
                    canvas.width.toFloat(),
                    canvas.height.toFloat(),
                    this,
                )
            }
        }
        PhotoFramePreset.CINEMA -> {
            canvas.drawColor(Color.rgb(150, 3, 9, 15))
            Paint(Paint.ANTI_ALIAS_FLAG).apply {
```



```

        shader = LinearGradient(
            0f,
            canvas.height * 0.62f,
            0f,
            canvas.height.toFloat(),
            Color.argb(0, 0, 0, 0),
            Color.argb(110, 0, 0, 0),
            Shader.TileMode.CLAMP,
        )
        canvas.drawRect(
            0f,
            canvas.height * 0.60f,
            canvas.width.toFloat(),
            canvas.height.toFloat(),
            this,
        )
    }
}

PhotoFramePreset.FROSTED -> {
    // 保留照片主色的同时覆盖一层冷白雾面，让整块外框像透过磨砂玻璃；
    // 参数区还会叠一层独立玻璃胶囊，形成清晰的材质层级。
    Paint(Paint.ANTI_ALIAS_FLAG).apply {
        shader = LinearGradient(
            0f,
            0f,
            0f,
            canvas.height.toFloat(),
            Color.argb(92, 250, 253, 255),
            Color.argb(132, 231, 239, 245),
            Shader.TileMode.CLAMP,
        )
        canvas.drawRect(
            0f,
            0f,
            canvas.width.toFloat(),
            canvas.height.toFloat(),
            this,
        )
    }
}

PhotoFramePreset.MINIMAL -> Unit
PhotoFramePreset.PLAQUE -> Unit
}

PhotoFramePreset.PLAQUE -> canvas.drawColor(Color.WHITE)
}

}

/**
 * 小图两阶段滑动窗口盒式模糊。只在最多 192×192 的背景代理图上执行，
 * 两轮耗时很小且内存固定；alpha 始终按不透明处理，避免边缘发黑。

```

```
*/
private fun blurBitmapInPlace(bitmap: Bitmap, radius: Int, passes: Int) {
    val width = bitmap.width
    val height = bitmap.height
    var source = IntArray(width * height)
    var target = IntArray(source.size)
    bitmap.getPixels(source, 0, width, 0, 0, width, height)
    val diameter = radius * 2 + 1
    repeat(passes) {
        for (y in 0 until height) {
            var red = 0L
            var green = 0L
            var blue = 0L
            for (offset in -radius..radius) {
                val color = source[y * width + offset.coerceIn(0, width - 1)]
                red += (color ushr 16) and 0xFF
                green += (color ushr 8) and 0xFF
                blue += color and 0xFF
            }
            for (x in 0 until width) {
                target[y * width + x] = Color.rgb(
                    (red / diameter).toInt(),
                    (green / diameter).toInt(),
                    (blue / diameter).toInt(),
                )
                val leaving = source[y * width + (x - radius).coerceIn(0, width - 1)]
                val entering = source[y * width + (x + radius + 1).coerceIn(0, width - 1)]
                red += ((entering ushr 16) and 0xFF) - ((leaving ushr 16) and 0xFF)
                green += ((entering ushr 8) and 0xFF) - ((leaving ushr 8) and 0xFF)
                blue += (entering and 0xFF) - (leaving and 0xFF)
            }
        }
        source = target.also { target = source }
        for (x in 0 until width) {
            var red = 0L
            var green = 0L
            var blue = 0L
            for (offset in -radius..radius) {
                val color = source[offset.coerceIn(0, height - 1) * width + x]
                red += (color ushr 16) and 0xFF
                green += (color ushr 8) and 0xFF
                blue += color and 0xFF
            }
            for (y in 0 until height) {
                target[y * width + x] = Color.rgb(
                    (red / diameter).toInt(),
                    (green / diameter).toInt(),
                    (blue / diameter).toInt(),
                )
                val leaving = source[(y - radius).coerceIn(0, height - 1) * width + x]
```

```

        val entering = source[(y + radius + 1).coerceIn(0, height - 1) * width + x]
        red += ((entering ushr 16) and 0xFF) - ((leaving ushr 16) and 0xFF)
        green += ((entering ushr 8) and 0xFF) - ((leaving ushr 8) and 0xFF)
        blue += (entering and 0xFF) - (leaving and 0xFF)
    }
}
source = target.also { target = source }
}
bitmap.setPixels(source, 0, width, 0, 0, width, height)
}
private fun Canvas.drawCenterCrop(bitmap: Bitmap, target: RectF) {
    val scale = maxOf(target.width() / bitmap.width, target.height() / bitmap.height)
    val width = bitmap.width * scale
    val height = bitmap.height * scale
    val rect = RectF(
        target.centerX() - width / 2f,
        target.centerY() - height / 2f,
        target.centerX() + width / 2f,
        target.centerY() + height / 2f,
    )
    drawBitmap(bitmap, null, rect, Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG))
}
private fun drawMetadata(
    context: Context,
    canvas: Canvas,
    layout: PhotoFrameLayout,
    metadata: PhotoFrameMetadata,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark,
) {
    // 图内水印已在照片裁剪区域中绘制，不能再作为边框信息区的一行参与排版。
    val metadataWatermark = watermark.withoutPhotoPlacement()
    val lightText =
        preset == PhotoFramePreset.MIST || preset == PhotoFramePreset.CINEMA
    val textColor =
        if (lightText) Color.rgb(248, 250, 252) else Color.rgb(25, 31, 38)
    val mutedColor =
        if (lightText) Color.rgb(220, 227, 233) else Color.rgb(70, 79, 88)
    val brand = normalizeCameraMake(metadata.make)
    val model = normalizeCameraModel(metadata.make, metadata.model)
    val details = frameDetailLine(metadata)
    val hasTitle = brand.isNotEmpty() || model.isNotEmpty()
    val hasDetails = details.isNotBlank()
    val centerX = layout.canvasWidth / 2f
    val contentArea = if (preset == PhotoFramePreset.FROSTED) {
        frostedMetadataPanelBounds(layout)
    } else {
        // 真正可见的下边框从照片底边开始；metadataTop 只是排版预留线，在部分长宽比
        // 下会比照片底边更低，用它居中正是旧版文字看起来整体偏下的根源。
        RectF(

```

```
        Of,
        layout.photoBottom,
        layout.canvasWidth.toFloat(),
        layout.canvasHeight.toFloat(),
    )
}
val brandPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
    color = textColor
    textSize = layout.canvasWidth * 0.032f
    typeface = Typeface.create("sans-serif", Typeface.BOLD_ITALIC)
}
val modelPaint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
    color = textColor
    textSize = layout.canvasWidth * 0.024f
    typeface = Typeface.create("sans-serif", Typeface.NORMAL)
}
var gap = if (brand.isNotEmpty() && model.isNotEmpty()) {
    layout.canvasWidth * 0.016f
} else {
    Of
}
val initialWidth = brandPaint.measureText(brand) + gap + modelPaint.measureText(model)
val maxTitleWidth = layout.canvasWidth *
    if (preset == PhotoFramePreset.FROSTED) 0.78f else 0.86f
if (initialWidth > maxTitleWidth) {
    val scale = maxTitleWidth / initialWidth
    brandPaint.textSize *= scale
    modelPaint.textSize *= scale
    gap *= scale
}
val detailPaint = if (hasDetails) {
    val paint = Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = mutedColor
        textSize = layout.canvasWidth * 0.020f
        typeface = Typeface.create("sans-serif", Typeface.NORMAL)
        textAlign = Paint.Align.CENTER
    }
    val maxDetailWidth = layout.canvasWidth *
        if (preset == PhotoFramePreset.FROSTED) 0.76f else 0.82f
    val detailWidth = paint.measureText(details)
    if (detailWidth > maxDetailWidth) {
        paint.textSize *= maxDetailWidth / detailWidth
    }
    paint
} else {
    null
}
val watermarkText = metadataWatermark.displayText
val watermarkPaint = if (metadataWatermark.enabled) {
    createWatermarkPaint(
```

```
        context = context,
        canvas = canvas,
        preset = preset,
        watermark = metadataWatermark,
        maxWidth = contentArea.width() * 0.86f,
    )
} else {
    null
}
fun currentRows(): Triple<FrameTextVisualBounds?, FrameTextVisualBounds?, FrameTextVisualBounds?> {
    val title = if (hasTitle) {
        listOfNotNull(
            brand.takeIf(String::isEmpty)??.let { textVisualBounds(it, brandPaint) },
            model.takeIf(String::isEmpty)??.let { textVisualBounds(it, modelPaint) },
        ).reduce(::mergeTextVisualBounds)
    } else {
        null
    }
    return Triple(
        title,
        detailPaint?.let { textVisualBounds(details, it) },
        watermarkPaint?.let { textVisualBounds(watermarkText, it) },
    )
}
var (titleBounds, detailBounds, watermarkBounds) = currentRows()
val initialRows = listOfNotNull(titleBounds, detailBounds, watermarkBounds)
if (initialRows.isEmpty()) return
if (preset == PhotoFramePreset.FROSTED) {
    drawFrostedMetadataPanel(canvas, layout, contentArea)
}
val rowScale = frameTextScaleToFit(contentArea.height(), initialRows)
if (rowScale < 1f) {
    brandPaint.textSize *= rowScale
    modelPaint.textSize *= rowScale
    detailPaint?.let { it.textSize *= rowScale }
    watermarkPaint?.let { it.textSize *= rowScale }
    currentRows().let { fitted ->
        titleBounds = fitted.first
        detailBounds = fitted.second
        watermarkBounds = fitted.third
    }
}
val rowBounds = listOfNotNull(titleBounds, detailBounds, watermarkBounds)
val preferredGap = min(
    layout.canvasWidth * 0.0125f,
    contentArea.height() * 0.09f,
)
val baselines = centeredFrameTextBaselines(
    areaTop = contentArea.top,
    areaBottom = contentArea.bottom,
```

```

        rows = rowBounds,
        preferredGap = preferredGap,
    )
    var rowIndex = 0
    val titleBaseline = if (titleBounds != null) baselines[rowIndex++] else null
    val detailBaseline = if (detailBounds != null) baselines[rowIndex++] else null
    val watermarkBaseline = if (watermarkBounds != null) baselines[rowIndex] else null
    titleBaseline?.let { baseline ->
        val totalWidth =
            brandPaint.measureText(brand) + gap + modelPaint.measureText(model)
        var x = centerX - totalWidth / 2f
        if (brand.isNotEmpty()) {
            canvas.drawText(brand, x, baseline, brandPaint)
            x += brandPaint.measureText(brand) + gap
        }
        if (model.isNotEmpty()) {
            canvas.drawText(model, x, baseline, modelPaint)
        }
    }
    if (detailPaint != null && detailBaseline != null) {
        canvas.drawText(details, centerX, detailBaseline, detailPaint)
    }
    if (watermarkPaint != null && watermarkBaseline != null) {
        val (x, align) = watermarkHorizontalPlacement(
            contentArea,
            preset,
            metadataWatermark.position,
        )
        watermarkPaint.textAlign = align
        drawWatermarkText(
            canvas,
            watermarkText,
            x,
            watermarkBaseline,
            watermarkPaint,
            metadataWatermark,
        )
    }
}

/** 不创建外框画布，只按原照片比例缩放并在画面安全区内叠加水印。 */
private fun renderWatermarkOnly(
    context: Context,
    source: Bitmap,
    watermark: PhotoFrameWatermark,
    longEdge: Int,
): Bitmap {
    val scale = min(1f, longEdge.toFloat() / maxOf(source.width, source.height))
    val width = (source.width * scale).roundToInt().coerceAtLeast(1)
    val height = (source.height * scale).roundToInt().coerceAtLeast(1)
    val output = Bitmap.createBitmap(width, height, Bitmap.Config.ARGB_8888)

```

```
try {
    val canvas = Canvas(output)
    val photoRect = RectF(0f, 0f, width.toFloat(), height.toFloat())
    canvas.drawBitmap(
        source,
        null,
        photoRect,
        Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG or Paint.DITHER_FLAG),
    )
    // 无边框时“自适应”固定使用带阴影的亮色方案，不受隐藏的边框预设影响。
    drawPhotoWatermark(
        context,
        canvas,
        photoRect,
        PhotoFramePreset.MIST,
        watermark,
    )
    return output
} catch (error: Throwable) {
    output.recycle()
    throw error
}
}

private fun drawFrostedMetadataPanel(
    canvas: Canvas,
    layout: PhotoFrameLayout,
    panel: RectF,
) {
    val radius = photoFrameCornerRadius(layout)
    Paint(Paint.ANTI_ALIAS_FLAG).apply {
        color = Color.argb(150, 250, 253, 255)
        setShadowLayer(
            layout.canvasWidth * 0.009f,
            0f,
            layout.canvasHeight * 0.004f,
            Color.argb(52, 20, 35, 46),
        )
        canvas.drawRoundRect(panel, radius, radius, this)
        clearShadowLayer()
    }
    Paint(Paint.ANTI_ALIAS_FLAG).apply {
        style = Paint.Style.STROKE
        strokeWidth = maxOf(1f, layout.canvasWidth * 0.0011f)
        color = Color.argb(178, 255, 255, 255)
        canvas.drawRoundRect(panel, radius, radius, this)
    }
}

private fun frostedMetadataPanelBounds(layout: PhotoFrameLayout): RectF {
    val bandHeight = layout.canvasHeight - layout.photoBottom
    val horizontalInset = layout.canvasWidth * 0.072f
```

```

        val verticalInset = bandHeight * 0.08f
        return RectF(
            horizontalInset,
            layout.photoBottom + verticalInset,
            layout.canvasWidth - horizontalInset,
            layout.canvasHeight - verticalInset,
        )
    }
}

private fun createWatermarkPaint(
    context: Context,
    canvas: Canvas,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark,
    maxWidth: Float,
): Paint {
    val shortEdge = min(canvas.width, canvas.height).toFloat()
    return Paint(Paint.ANTI_ALIAS_FLAG or Paint.SUBPIXEL_TEXT_FLAG).apply {
        color = when (watermark.color) {
            // 显式颜色保持低饱和与轻透明，作为照片署名而不是浮在画面上的贴纸。
            PhotoFrameWatermarkColor.WHITE -> Color.rgb(244, 239, 228)
            PhotoFrameWatermarkColor.BLACK -> Color.rgb(50, 55, 60)
            PhotoFrameWatermarkColor.GOLD -> Color.rgb(204, 172, 112)
            PhotoFrameWatermarkColor.MIST_BLUE -> Color.rgb(132, 157, 180)
            PhotoFrameWatermarkColor.ROSE_GOLD -> Color.rgb(185, 128, 121)
            PhotoFrameWatermarkColor.ADAPTIVE -> when (preset) {
                PhotoFramePreset.MIST,
                PhotoFramePreset.CINEMA -> Color.rgb(250, 252, 253)
                PhotoFramePreset.MINIMAL,
                PhotoFramePreset.FROSTED -> Color.rgb(24, 31, 38)
                PhotoFramePreset.PLAQUE -> Color.rgb(24, 31, 38)
            }
        }
        alpha = watermarkAlpha(watermark.opacityPercent)
        // 100% 对应旧“大号”，200% 即旧最大尺寸的两倍。
        textSize = shortEdge * photoFrameWatermarkTextSizeFraction(watermark.sizePercent)
        typeface = when (watermark.font) {
            PhotoFrameWatermarkFont.SIGNATURE,
            PhotoFrameWatermarkFont.ELEGANT,
            PhotoFrameWatermarkFont.CALLIGRAPHY ->
                bundledWatermarkTypeface(context, watermark.font)
            PhotoFrameWatermarkFont.SIMPLE ->
                Typeface.create("sans-serif", Typeface.NORMAL)
            PhotoFrameWatermarkFont.BOLD ->
                Typeface.create("sans-serif", Typeface.BOLD)
        }
        textAlign = Paint.Align.CENTER
        val measured = measureText(watermark.displayText)
        if (measured > maxWidth && measured > 0f) {
            textSize *= maxWidth / measured
        }
    }
}

```



```

    }
}
/**
 * 三款艺术字体随 APK 离线分发并缓存 Typeface，避免预览连续重绘时重复解析字体文件。
 * 资源异常时仍回退到系统字体，不能让一张分享图因为字体加载失败而中断导出。
 */
private fun bundledWatermarkTypeface(
    context: Context,
    font: PhotoFrameWatermarkFont,
): Typeface = synchronized(bundledTypefaceCache) {
    bundledTypefaceCache.getOrPut(font) {
        val resourceId = when (font) {
            PhotoFrameWatermarkFont.SIGNATURE -> R.font.great_vibes_regular
            PhotoFrameWatermarkFont.ELEGANT -> R.font.cormorant_garamond_medium_italic
            PhotoFrameWatermarkFont.CALLIGRAPHY -> R.font.bebas_neue_regular
            PhotoFrameWatermarkFont.SIMPLE,
            PhotoFrameWatermarkFont.BOLD -> error("System font does not use bundled resources")
        }
        ResourcesCompat.getFont(context.applicationContext, resourceId)
        ?: Typeface.create("serif", Typeface.NORMAL)
    }
}
private fun drawPhotoWatermark(
    context: Context,
    canvas: Canvas,
    photoRect: RectF,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark,
) {
    if (!watermark.enabled || !watermark.position.isPhotoPlacement()) return
    val safeInset = min(photoRect.width(), photoRect.height()) * 0.04f
    if (watermark.content == PhotoFrameWatermarkContent.IMAGE) {
        drawPhotoImageWatermark(
            context = context,
            canvas = canvas,
            photoRect = photoRect,
            safeInset = safeInset,
            watermark = watermark,
        )
        return
    }
    val paint = createWatermarkPaint(
        context = context,
        canvas = canvas,
        preset = preset,
        watermark = watermark,
        maxWidth = (photoRect.width() - safeInset * 2f).coerceAtLeast(1f),
    ).apply {
        // The placement calculation uses the actual glyph bounds from a left-aligned origin.
        textAlign = Paint.Align.LEFT
    }
}

```

```
}
val text = watermark.displayText
val bounds = Rect().also { paint.getTextBounds(text, 0, text.length, it) }
val placement = calculatePhotoWatermarkPlacement(
    photoLeft = photoRect.left,
    photoTop = photoRect.top,
    photoRight = photoRect.right,
    photoBottom = photoRect.bottom,
    textBounds = PhotoWatermarkTextBounds(
        left = bounds.left.toFloat(),
        top = bounds.top.toFloat(),
        right = bounds.right.toFloat(),
        bottom = bounds.bottom.toFloat(),
    ),
    position = watermark.position,
)
drawWatermarkText(canvas, text, placement.originX, placement.baseline, paint, watermark)
}
private fun drawPhotoImageWatermark(
    context: Context,
    canvas: Canvas,
    photoRect: RectF,
    safeInset: Float,
    watermark: PhotoFrameWatermark,
) {
    val imageHash = requireNotNull(validPhotoFrameWatermarkImageHash(watermark.imageHash)) {
        "Image watermark has no valid private copy"
    }
    val bitmap = loadWatermarkImage(context, imageHash)
    val shortEdge = min(photoRect.width(), photoRect.height())
    // 图片与文字采用同一百分比语义: 100% 是旧“大号”, 最大可到旧值两倍。
    var targetHeight = shortEdge *
        photoFrameWatermarkImageSizeFraction(watermark.sizePercent)
    var targetWidth = targetHeight * bitmap.width / bitmap.height.toFloat()
    val maxWidth = (photoRect.width() - safeInset * 2f).coerceAtLeast(1f)
    if (targetWidth > maxWidth) {
        val scale = maxWidth / targetWidth
        targetWidth *= scale
        targetHeight *= scale
    }
    val bounds = PhotoWatermarkTextBounds(
        left = 0f,
        top = -targetHeight,
        right = targetWidth,
        bottom = 0f,
    )
    val placement = calculatePhotoWatermarkPlacement(
        photoLeft = photoRect.left,
        photoTop = photoRect.top,
        photoRight = photoRect.right,
```

```
        photoBottom = photoRect.bottom,
        textBounds = bounds,
        position = watermark.position,
    )
    val destination = RectF(
        placement.originX,
        placement.baseline - targetHeight,
        placement.originX + targetWidth,
        placement.baseline,
    )
    drawDownsampledWatermarkBitmap(
        canvas = canvas,
        source = bitmap,
        destination = destination,
        alpha = watermarkAlpha(watermark.opacityPercent),
    )
}
/**
 * Logo 通常带有细线和透明边缘。一次把大图缩到几十像素会跨过过多源像素，导致细线
 * 呈断续状；逐级减半相当于建立临时 mip 层，再完成最后一次缩放。源文件和缓存位图
 * 始终不变，预览与导出共用这一采样路径。
 */
private fun drawDownsampledWatermarkBitmap(
    canvas: Canvas,
    source: Bitmap,
    destination: RectF,
    alpha: Int,
) {
    val targetWidth = destination.width().roundToInt().coerceAtLeast(1)
    val targetHeight = destination.height().roundToInt().coerceAtLeast(1)
    var sampled = source
    try {
        while (sampled.width > targetWidth * 2 && sampled.height > targetHeight * 2) {
            val next = Bitmap.createScaledBitmap(
                sampled,
                maxOf(targetWidth, sampled.width / 2),
                maxOf(targetHeight, sampled.height / 2),
                true,
            )
            if (sampled != source && sampled != next) sampled.recycle()
            sampled = next
        }
        canvas.drawBitmap(
            sampled,
            null,
            destination,
            Paint(
                Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG or Paint.DITHER_FLAG,
            ).apply {
                this.alpha = alpha
            }
        )
    }
```

```

        },
    )
} finally {
    if (sampled != source) sampled.recycle()
}
}

private fun loadWatermarkImage(context: Context, imageHash: String): Bitmap =
    synchronized(watermarkImageCache) {
        watermarkImageCache[imageHash]?.let { return@synchronized it }
        val file = photoFrameWatermarkImageFile(context.applicationContext, imageHash)
        require(file.isFile) { "Private watermark image is missing" }
        val bounds = BitmapFactory.Options().apply { inJustDecodeBounds = true }
        BitmapFactory.decodeFile(file.absolutePath, bounds)
        require(bounds.outWidth > 0 && bounds.outHeight > 0) {
            "Private watermark image cannot be decoded"
        }
        var sampleSize = 1
        while (maxOf(bounds.outWidth, bounds.outHeight) / sampleSize > 2048) {
            sampleSize *= 2
        }
        val bitmap = BitmapFactory.decodeFile(
            file.absolutePath,
            BitmapFactory.Options().apply { inSampleSize = sampleSize },
        ) ?: error("Private watermark image cannot be decoded")
        watermarkImageCache[imageHash] = bitmap
        while (watermarkImageCache.size > 3) {
            watermarkImageCache.remove(watermarkImageCache.keys.first())
        }
        bitmap
    }

private fun drawWatermarkText(
    canvas: Canvas,
    text: String,
    x: Float,
    baseline: Float,
    paint: Paint,
    watermark: PhotoFrameWatermark,
) {
    when (resolvedWatermarkEffect(watermark)) {
        PhotoFrameWatermarkEffect.NONE -> canvas.drawText(text, x, baseline, paint)
        PhotoFrameWatermarkEffect.SHADOW -> {
            paint.setShadowLayer(
                paint.textSize * 0.12f,
                0f,
                paint.textSize * 0.06f,
                contrastingWatermarkColor(paint.color, paint.alpha),
            )
            canvas.drawText(text, x, baseline, paint)
            paint.clearShadowLayer()
        }
    }
}

```

```

        PhotoFrameWatermarkEffect.OUTLINE -> {
            val outline = Paint(paint).apply {
                style = Paint.Style.STROKE
                strokeWidth = maxOf(1f, paint.textSize * 0.075f)
                strokeJoin = Paint.Join.ROUND
                color = contrastingWatermarkColor(paint.color, paint.alpha)
            }
            canvas.drawText(text, x, baseline, outline)
            canvas.drawText(text, x, baseline, paint)
        }
        PhotoFrameWatermarkEffect.AUTO -> error("AUTO must be resolved before drawing")
    }
}

private fun watermarkHorizontalPlacement(
    area: RectF,
    preset: PhotoFramePreset,
    requested: PhotoFrameWatermarkPosition,
): Pair<Float, Paint.Align> {
    val position = resolvedWatermarkPosition(preset, requested)
    val inset = area.width() * if (preset == PhotoFramePreset.PLAQUE) 0.058f else 0.07f
    return when (position) {
        PhotoFrameWatermarkPosition.LEFT -> area.left + inset to Paint.Align.LEFT
        PhotoFrameWatermarkPosition.RIGHT -> area.right - inset to Paint.Align.RIGHT
        PhotoFrameWatermarkPosition.CENTER,
        PhotoFrameWatermarkPosition.AUTO -> area.centerX() to Paint.Align.CENTER
        else -> error("Photo placement must not be drawn in the metadata area")
    }
}

private fun textVisualBounds(text: String, paint: Paint): FrameTextVisualBounds {
    val bounds = Rect()
    paint.getTextBounds(text, 0, text.length, bounds)
    return FrameTextVisualBounds(bounds.top.toFloat(), bounds.bottom.toFloat())
}

private fun mergeTextVisualBounds(
    first: FrameTextVisualBounds,
    second: FrameTextVisualBounds,
): FrameTextVisualBounds = FrameTextVisualBounds(
    top = min(first.top, second.top),
    bottom = maxOf(first.bottom, second.bottom),
)

/**
 * “铭牌”预设严格沿用参考图的结构：照片无裁切铺满宽度，底部接一整条白色信息带。
 * 横图查看时出现的上下黑区属于图库查看器，不写进导出文件。
 */
private fun drawPlaqueFrame(
    context: Context,
    canvas: Canvas,
    source: Bitmap,
    layout: PhotoFrameLayout,
    metadata: PhotoFrameMetadata,

```

```
        watermark: PhotoFrameWatermark,
    ) {
        val metadataWatermark = watermark.withoutPhotoPlacement()
        canvas.drawColor(Color.WHITE)
        val photoRect = RectF(
            layout.photoLeft,
            layout.photoTop,
            layout.photoRight,
            layout.photoBottom,
        )
        canvas.drawBitmap(
            source,
            null,
            photoRect,
            Paint(Paint.ANTI_ALIAS_FLAG or Paint.FILTER_BITMAP_FLAG or Paint.DITHER_FLAG),
        )
        drawPhotoWatermark(context, canvas, photoRect, PhotoFramePreset.PLAQUE, watermark)
        val width = layout.canvasWidth.toFloat()
        val bandTop = layout.metadataTop
        val bandHeight = layout.canvasHeight - bandTop
        Paint(Paint.ANTI_ALIAS_FLAG).apply {
            color = Color.rgb(253, 253, 252)
            canvas.drawRect(0f, bandTop, width, layout.canvasHeight.toFloat(), this)
            color = Color.rgb(232, 233, 231)
            canvas.drawRect(
                0f,
                bandTop,
                width,
                bandTop + maxOf(1f, width * 0.0008f),
                this,
            )
        }
        val make = metadata.make?.trim()
            ?.takeIf(String::isNotEmpty)
            ?.uppercase(Locale.ROOT)
        val model = metadata.model?.trim()?.takeIf(String::isNotEmpty)
        val details = frameDetailLine(metadata).takeIf(String::isNotEmpty)
        val date = metadata.dateTime?.takeIf(String::isNotEmpty)
        val leftPrimary = make ?: model
        val leftSecondary = model?.takeIf { make != null && !it.equals(make, ignoreCase = true) }
        val rightPrimary = details ?: date
        val rightSecondary = date?.takeIf { details != null }
        val hasLeft = leftPrimary != null
        val hasRight = rightPrimary != null
        val hasLeftBlock = hasLeft
        val leftX = width * 0.058f
        val leftMaxWidth = width * if (hasRight) 0.46f else 0.884f
        val rightX = width * if (hasLeftBlock) 0.60f else 0.058f
        val rightMaxWidth = width * if (hasLeftBlock) 0.35f else 0.884f
        val leftPrimaryPaint = leftPrimary?.let {
```

```

}
/** 兼容旧版 SUBTLE/STANDARD/STRONG 字符串；新版本直接持久化百分比。 */
internal fun restoredPhotoFrameWatermarkOpacityPercent(persisted: Any?): Int {
    val rawPercent = when (persisted) {
        is Number -> persisted.toInt()
        is String -> persisted.toIntOrNull() ?: when (persisted) {
            "SUBTLE" -> 40
            "STANDARD" -> 72
            "STRONG" -> 100
            else -> DEFAULT_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT
        }
        else -> DEFAULT_PHOTO_FRAME_WATERMARK_OPACITY_PERCENT
    }
    return normalizePhotoFrameWatermarkOpacityPercent(rawPercent)
}

internal val TransferState.photoFrameWatermark: PhotoFrameWatermark
    get() = PhotoFrameWatermark(
        enabled = photoFrameWatermarkEnabled,
        content = photoFrameWatermarkContent,
        text = photoFrameWatermarkText,
        imageHash = photoFrameWatermarkImageHash,
        font = photoFrameWatermarkFont,
        sizePercent = photoFrameWatermarkSizePercent,
        position = photoFrameWatermarkPosition,
        color = photoFrameWatermarkColor,
        opacityPercent = photoFrameWatermarkOpacityPercent,
        effect = photoFrameWatermarkEffect,
    )

/** 队列剩余待处理数量（正在传、等待传或正在生成派生图）。供顶栏药丸等复用。 */
val TransferState.remainingCount: Int
    get() = tasks.count {
        it.status == TransferStatus.WAITING ||
        it.status == TransferStatus.TRANSFERING ||
        it.isGeneratingFrame
    }

/**
 * 当前处理项的进度（0..1）。派生图没有廉价而准确的分段进度，原片已完整落盘后保持满格，
 * 直到派生完成；这样顶栏不会在最后一步从 100% 倒退到 0%。
 */
val TransferState.currentFileProgress: Float
    get() = tasks.firstOrNull { it.status == TransferStatus.TRANSFERING }?.progress
        ?: if (tasks.any { it.isGeneratingFrame }) 1f else 0f

/** 免费版固定使用默认水印；高级版完整采用用户设置。预览和真正导出必须共用该入口。 */
internal fun effectivePhotoFrameWatermark(
    isPro: Boolean,
    preference: PhotoFrameWatermark,
    borderEnabled: Boolean = true,
): PhotoFrameWatermark {
    val permitted = if (isPro) preference else PhotoFrameWatermark()
    val imageHash = validPhotoFrameWatermarkImageHash(permitted.imageHash)

```

```

    val content = if (
        permitted.content == PhotoFrameWatermarkContent.IMAGE && imageHash != null
    ) {
        PhotoFrameWatermarkContent.IMAGE
    } else {
        PhotoFrameWatermarkContent.TEXT
    }
    return permitted.copy(
        content = content,
        text = permitted.displayText,
        imageHash = imageHash,
        sizePercent = normalizePhotoFrameWatermarkSizePercent(permitted.sizePercent),
        opacityPercent = normalizePhotoFrameWatermarkOpacityPercent(permitted.opacityPercent),
        position = if ((!borderEnabled || content == PhotoFrameWatermarkContent.IMAGE) &&
            !permitted.position.isPhotoPlacement())
        ) {
            PhotoFrameWatermarkPosition.PHOTO_BOTTOM_RIGHT
        } else {
            permitted.position
        },
    )
}

/** 只允许 JPG/JPEG/PNG 派生分享图；视频、RAW 和未知类型始终保持原样传输。 */
internal fun shouldGeneratePhotoFrame(enabled: Boolean, extension: String): Boolean =
    enabled && isSupportedPhotoFrameSourceExtension(extension)

/** 相机文件是否已在当前保存目录中落盘；列表对号、筛选和任务模式必须共用该判定。 */
internal fun isTransferredOriginal(
    file: NikonCamera.FileInfo,
    existingExportFiles: Map<String, Set<Long>>,
): Boolean {
    val sizes = existingExportFiles[file.fileName] ?: return false
    return sizes.any { it < 0L || it == file.size } ||
        file.size == PtpConstants.SIZE_UNKNOWN
}

internal fun createQueueTasks(
    files: List<NikonCamera.FileInfo>,
    photoFrameEnabled: Boolean,
    photoFrameBorderEnabled: Boolean = true,
    photoFramePreset: PhotoFramePreset,
    photoFrameWatermark: PhotoFrameWatermark,
): List<TransferTask> = files.asSequence()
    // 同一次批量点击按相机文件去重；不同点击始终创建独立任务。
    .distinctBy { it.handle }
    .map { file ->
        TransferTask(
            file = file,
            framePreset = photoFramePreset.takeIf {
                shouldGeneratePhotoFrame(photoFrameEnabled, file.extension)
            },
            frameBorderRequested = photoFrameBorderEnabled,

```



```

        frameWatermarkRequested = photoFrameWatermark,
    )
}
.toList()
class TransferViewModel(application: Application) : AndroidViewModel(application) {
    private val _state = MutableStateFlow(TransferState())
    val state: StateFlow<TransferState> = _state.asStateFlow()
    private var transferJob: Job? = null
    private val prefs = application.getSharedPreferences("ztransfer", Context.MODE_PRIVATE)
    private val contentResolver = application.contentResolver
    /**
     * 边框渲染使用独立、低优先级、单并发线程：与下一张相机传输并行，但不会同时展开
     * 多张 3200px 位图争抢内存。相机网络/USB 传输仍留在原来的高优先级通道。
     */
    private val photoFrameDispatcher = Executors.newSingleThreadExecutor { task ->
        Thread(
            {
                android.os.Process.setThreadPriority(android.os.Process.THREAD_PRIORITY_BACKGROUND)
                task.run()
            },
            "photo-frame-export",
        )
    }.asCoroutineDispatcher()
    private val activePhotoFrameExports = AtomicInteger(0)
    // 第一张派生图才创建/扫描专用子目录；同一根目录后续任务复用，避免逐张遍历文件夹。
    private val photoFrameDestinations =
        ConcurrentHashMap<String, PhotoFrameDestination>()
    /** 用户可见文案（错误信息等）统一走字符串资源；经 AppLocale.wrap 与应用内语言一致。 */
    private fun str(resId: Int, vararg args: Any?): String =
        AppLocale.wrap(getApplication()).getString(resId, *args)
    /**
     * 把底层异常翻译成用户可读的三语文案：网络类异常（断联/超时/连接重置——卡片上
     * 曾裸露 "software caused connection abort" 这类系统原文）统一显示"相机连接中断"；
     * 目录失效单独指认；其余保留自带信息（多为我们自己抛出的已本地化业务文案）。
     */
    private fun friendlyError(e: Throwable?): String {
        val msg = e?.message ?: return str(R.string.transfer_failed)
        val connectionLost = e is java.net.SocketException ||
            e is java.net.SocketTimeoutException ||
            e is java.io.EOFException ||
            listOf("connection abort", "connection reset", "broken pipe",
                "socket", "econn", "etimedout", "network is unreachable")
                .any { msg.contains(it, ignoreCase = true) }
        return when {
            connectionLost -> str(R.string.error_camera_connection_lost)
            e is java.io.FileNotFoundException -> str(R.string.error_dir_invalid)
            else -> msg
        }
    }
}
// 鸿蒙适配：部分华为/荣耀设备的 DocumentsProvider renameDocument 损坏（无论目标名

```

```

// 是否空闲都失败），下载完好的临时文件改不了正式名 → 每张都“保存失败”。
// 首次确认损坏后置位，本会话后续文件跳过改名直接走“复制为正式文件”回退路径，
// 不再每个文件白试上百次改名。安卓正常设备永远不会置位，行为零变化。
private var renameBroken = false
private companion object {
    const val TAG = "ZTransfer"
    // 未完成文件的临时名前缀（带前导点，在相册中隐藏）。真正文件名只在下载完整后才出现。
    const val PART_PREFIX = ".nkpart_"
    // 重副本后缀（"DSC_0001 (1).NEF" 中的 "(1)"），用于剥离/生成。
    val COPY_SUFFIX_REGEX = Regex("^(\\d+\\) (?=\\. [^\\.]*\$)"")
    // 分块大小引用协议层常量，保证断点续传偏移与分块下载粒度的严格一致。
    val RESUME_CHUNK_SIZE: Long get() = NikonCamera.CHUNK_SIZE
}
/** 半成品文件信息：用于断点续传。[token] = 文件内容身份（大小+拍摄时间），
 * 防止同名不同文件（DSC 编号跨文件夹回卷）续传时张冠李戴、把两份数据拼接成损坏文件。 */
private data class PartInfo(val uri: Uri, val size: Long, val token: String)
private data class ExistingDirectoryScan(
    val sizes: MutableMap<String, Long>,
    val uris: MutableMap<String, Uri>,
    val parts: Map<String, PartInfo>,
)
private data class LocalOriginal(
    val displayName: String,
    val size: Long,
    val uri: Uri,
)
private sealed interface FrameExportOutcome {
    data class Saved(val displayName: String) : FrameExportOutcome
    data class Failed(val error: Throwable) : FrameExportOutcome
    object AlreadyExists : FrameExportOutcome
}
/** 文件内容身份令牌：大小+拍摄时间，仅留字母数字与点（内嵌半成品名，不含下划线分隔符）。 */
private fun identityToken(file: NikonCamera.FileInfo): String =
    "${file.size}.${file.captureDate ?: "0"}.replace(Regex("[^A-Za-z0-9.]"), "")
/** 半成品文件名 = 前缀 + 身份令牌 + "_" + 原文件名（原名可含下划线，解析按【首个】下划线切分）。 */
private fun partFileName(file: NikonCamera.FileInfo): String =
    PART_PREFIX + identityToken(file) + "_" + file.fileName
init {
    val dir = prefs.getString("transfer_dir", null)
    val storedSkinName = prefs.getString("skin_preset", null)
    val restoredSkinPreset = if (storedSkinName == null) {
        SkinPreset.FROSTED_GLASS
    } else {
        SkinPreset.entries.firstOrNull { it.name == storedSkinName } ?: SkinPreset.TITANIUM
    }
    // 已下线的旧材质或异常值统一迁移到钛合金，并立即写回新枚举名。
    if (storedSkinName != null && storedSkinName != restoredSkinPreset.name) {
        prefs.edit().putString("skin_preset", restoredSkinPreset.name).apply()
    }
    val restoredWatermarkImageHash = validPhotoFrameWatermarkImageHash(

```

```

        prefs.getString("photo_frame_watermark_image_hash", null)
    )?.takeIf { hash -> photoFrameWatermarkImageFile(application, hash).isFile }
    val restoredWatermarkContent = runCatching {
        PhotoFrameWatermarkContent.valueOf(
            prefs.getString(
                "photo_frame_watermark_content",
                PhotoFrameWatermarkContent.TEXT.name,
            ) ?: PhotoFrameWatermarkContent.TEXT.name,
        )
    }
    }.getOrDefault(PhotoFrameWatermarkContent.TEXT).let { content ->
        if (content == PhotoFrameWatermarkContent.IMAGE && restoredWatermarkImageHash == null) {
            PhotoFrameWatermarkContent.TEXT
        } else {
            content
        }
    }
}
val storedPreferences = prefs.all
val storedWatermarkSize = storedPreferences["photo_frame_watermark_size"]
val restoredWatermarkSizePercent = restoredPhotoFrameWatermarkSizePercent(
    storedWatermarkSize,
    restoredWatermarkContent,
)
val storedWatermarkOpacity = storedPreferences["photo_frame_watermark_opacity"]
val restoredWatermarkOpacityPercent =
    restoredPhotoFrameWatermarkOpacityPercent(storedWatermarkOpacity)
val sizeNeedsMigration = storedWatermarkSize != null &&
    (storedWatermarkSize !is Number ||
        storedWatermarkSize.toInt() != restoredWatermarkSizePercent)
val opacityNeedsMigration = storedWatermarkOpacity != null &&
    (storedWatermarkOpacity !is Number ||
        storedWatermarkOpacity.toInt() != restoredWatermarkOpacityPercent)
if (sizeNeedsMigration || opacityNeedsMigration) {
    prefs.edit()
        .putInt("photo_frame_watermark_size", restoredWatermarkSizePercent)
        .putInt("photo_frame_watermark_opacity", restoredWatermarkOpacityPercent)
        .apply()
}
_state.update {
    it.copy(
        transferDirUri = dir,
        thumbnailColumns = normalizeThumbnailColumns(
            prefs.getInt("thumbnail_columns", 3)
        ),
        collapseBurstPhotos = prefs.getBoolean("collapse_burst_photos", true),
        tapToPreview = prefs.getBoolean("tap_to_preview", false),
        hapticsEnabled = prefs.getBoolean("haptics_enabled", true),
        keepScreenOn = prefs.getBoolean("keep_screen_on", true),
        themeMode = prefs.getString("theme_mode", null)
            ?.let { m -> ThemeMode.entries.firstOrNull { e -> e.name == m } }
            ?: ThemeMode.SYSTEM,
    )
}

```

```
skinPreset = restoredSkinPreset,
// getStringSet 返回的实例不可直接持有 (SharedPreferences 约定), 拷贝一份。
filterExtensions = prefs.getStringSet("filter_exts", null)?.toSet(),
filterProtectedOnly = prefs.getBoolean("filter_protected", false),
filterBurstOnly = prefs.getBoolean("filter_burst", false),
filterUntransferredOnly = prefs.getBoolean("filter_untransferred", false),
previewRotationQuarterTurns = Math.floorMod(
    prefs.getInt("preview_rotation_quarter_turns", 0), 4
),
photoFrameEnabled = prefs.getBoolean("photo_frame_enabled", false),
photoFrameBorderEnabled = prefs.getBoolean("photo_frame_border_enabled", true),
photoFramePreset = runCatching {
    PhotoFramePreset.valueOf(
        prefs.getString("photo_frame_preset", PhotoFramePreset.MIST.name)
            ?: PhotoFramePreset.MIST.name
    )
}.getOrElse(PhotoFramePreset.MIST),
photoFrameWatermarkEnabled =
    prefs.getBoolean("photo_frame_branding_enabled", true),
photoFrameWatermarkContent = restoredWatermarkContent,
photoFrameWatermarkText =
    prefs.getString("photo_frame_watermark_text", "ZTransfer") ?: "ZTransfer",
photoFrameWatermarkImageHash = restoredWatermarkImageHash,
photoFrameWatermarkFont = runCatching {
    PhotoFrameWatermarkFont.valueOf(
        prefs.getString(
            "photo_frame_watermark_font",
            PhotoFrameWatermarkFont.ELEGANT.name,
        ) ?: PhotoFrameWatermarkFont.ELEGANT.name,
    )
}.getOrElse(PhotoFrameWatermarkFont.ELEGANT),
photoFrameWatermarkSizePercent = restoredWatermarkSizePercent,
photoFrameWatermarkPosition = runCatching {
    PhotoFrameWatermarkPosition.valueOf(
        prefs.getString(
            "photo_frame_watermark_position",
            PhotoFrameWatermarkPosition.AUTO.name,
        ) ?: PhotoFrameWatermarkPosition.AUTO.name,
    )
}.getOrElse(PhotoFrameWatermarkPosition.AUTO),
photoFrameWatermarkColor = runCatching {
    PhotoFrameWatermarkColor.valueOf(
        prefs.getString(
            "photo_frame_watermark_color",
            PhotoFrameWatermarkColor.ADAPTIVE.name,
        ) ?: PhotoFrameWatermarkColor.ADAPTIVE.name,
    )
}.getOrElse(PhotoFrameWatermarkColor.ADAPTIVE),
photoFrameWatermarkOpacityPercent = restoredWatermarkOpacityPercent,
photoFrameWatermarkEffect = runCatching {
```

```

        PhotoFrameWatermarkEffect.valueOf(
            prefs.getString(
                "photo_frame_watermark_effect",
                PhotoFrameWatermarkEffect.AUTO.name,
            ) ?: PhotoFrameWatermarkEffect.AUTO.name,
        )
    }.getOrDefault(PhotoFrameWatermarkEffect.AUTO),
    remoteRotation = prefs.getInt("remote_rotation", 0),
    appLanguage = prefs.getString(AppLocale.PREF_KEY, AppLocale.SYSTEM) ?: AppLocale.SYSTEM
)
}
// 开 App 时清扫上次崩溃/被杀留下的半成品 (.nkpart_ 临时文件)。
if (dir != null) {
    viewModelScope.launch(Dispatchers.IO) {
        refreshExistingExportFiles(Uri.parse(dir), deleteParts = true)
    }
}
}
fun setThumbnailColumns(columns: Int) {
    val c = normalizeThumbnailColumns(columns)
    prefs.edit().putInt("thumbnail_columns", c).apply()
    _state.update { it.copy(thumbnailColumns = c) }
}
fun setCollapseBurstPhotos(enabled: Boolean) {
    prefs.edit().putBoolean("collapse_burst_photos", enabled).apply()
    _state.update { it.copy(collapseBurstPhotos = enabled) }
}
fun setTapToPreview(enabled: Boolean) {
    prefs.edit().putBoolean("tap_to_preview", enabled).apply()
    _state.update { it.copy(tapToPreview = enabled) }
}
fun setThemeMode(mode: ThemeMode) {
    prefs.edit().putString("theme_mode", mode.name).apply()
    _state.update { it.copy(themeMode = mode) }
}
fun setSkinPreset(skin: SkinPreset) {
    prefs.edit().putString("skin_preset", skin.name).apply()
    _state.update { it.copy(skinPreset = skin) }
}
fun setHapticsEnabled(enabled: Boolean) {
    prefs.edit().putBoolean("haptics_enabled", enabled).apply()
    _state.update { it.copy(hapticsEnabled = enabled) }
}
fun setKeepScreenOn(enabled: Boolean) {
    prefs.edit().putBoolean("keep_screen_on", enabled).apply()
    _state.update { it.copy(keepScreenOn = enabled) }
}
}
/** 保存预览大图的全局旋转方向；任何照片和下次启动都复用。 */
fun setPreviewRotationQuarterTurns(turns: Int) {
    val normalized = Math.floorMod(turns, 4)

```

```
prefs.edit().putInt("preview_rotation_quarter_turns", normalized).apply()
_state.update { it.copy(previewRotationQuarterTurns = normalized) }
}
fun setPhotoFrameEnabled(enabled: Boolean) {
    val snapshot = _state.value
    val effectiveWatermark = effectivePhotoFrameWatermark(
        isPro = LicenseManager.isPro.value,
        preference = snapshot.photoFrameWatermark,
        borderEnabled = snapshot.photoFrameBorderEnabled,
    )
    val restoreBorder = enabled &&
        !snapshot.photoFrameBorderEnabled && !effectiveWatermark.enabled
    val borderEnabled = if (restoreBorder) true else snapshot.photoFrameBorderEnabled
    prefs.edit()
        .putBoolean("photo_frame_enabled", enabled)
        .putBoolean("photo_frame_border_enabled", borderEnabled)
        .apply()
    _state.update { state ->
        state.copy(
            photoFrameEnabled = enabled,
            photoFrameBorderEnabled = borderEnabled,
        )
    }
}
/**
 * 二级页退出时一次提交边框与水印，避免逐字写偏好或让主设置短暂显示半套新配置。
 * 免费版即使绕过 UI 传入水印也会被忽略，只允许更新边框样式。
 */
fun setPhotoFrameConfiguration(
    borderEnabled: Boolean,
    preset: PhotoFramePreset,
    watermark: PhotoFrameWatermark?,
) {
    val normalized = watermark
        ?.takeIf { LicenseManager.isPro.value }
        ?.let {
            effectivePhotoFrameWatermark(
                isPro = true,
                preference = it,
                borderEnabled = borderEnabled,
            )
        }
    val effectiveWatermark = normalized ?: effectivePhotoFrameWatermark(
        isPro = LicenseManager.isPro.value,
        preference = _state.value.photoFrameWatermark,
        borderEnabled = borderEnabled,
    )
    val decorationEnabled = borderEnabled || effectiveWatermark.enabled
    prefs.edit().apply {
        putBoolean("photo_frame_enabled", decorationEnabled)
    }
}
```

```
putBoolean("photo_frame_border_enabled", borderEnabled)
putString("photo_frame_preset", preset.name)
normalized?.let {
    // 沿用旧 key, 升级用户原有的水印开关偏好无需迁移。
    putBoolean("photo_frame_branding_enabled", it.enabled)
    putString("photo_frame_watermark_content", it.content.name)
    putString("photo_frame_watermark_text", it.text)
    if (it.imageHash == null) {
        remove("photo_frame_watermark_image_hash")
    } else {
        putString("photo_frame_watermark_image_hash", it.imageHash)
    }
    putString("photo_frame_watermark_font", it.font.name)
    putInt("photo_frame_watermark_size", it.sizePercent)
    putString("photo_frame_watermark_position", it.position.name)
    putString("photo_frame_watermark_color", it.color.name)
    putInt("photo_frame_watermark_opacity", it.opacityPercent)
    putString("photo_frame_watermark_effect", it.effect.name)
}
}.apply()
_state.update {
    it.copy(
        photoFrameEnabled = decorationEnabled,
        photoFrameBorderEnabled = borderEnabled,
        photoFramePreset = preset,
        photoFrameWatermarkEnabled = normalized?.enabled
            ?: it.photoFrameWatermarkEnabled,
        photoFrameWatermarkContent = normalized?.content
            ?: it.photoFrameWatermarkContent,
        photoFrameWatermarkText = normalized?.text
            ?: it.photoFrameWatermarkText,
        photoFrameWatermarkImageHash = if (normalized != null) {
            normalized.imageHash
        } else {
            it.photoFrameWatermarkImageHash
        },
        photoFrameWatermarkFont = normalized?.font
            ?: it.photoFrameWatermarkFont,
        photoFrameWatermarkSizePercent = normalized?.sizePercent
            ?: it.photoFrameWatermarkSizePercent,
        photoFrameWatermarkPosition = normalized?.position
            ?: it.photoFrameWatermarkPosition,
        photoFrameWatermarkColor = normalized?.color
            ?: it.photoFrameWatermarkColor,
        photoFrameWatermarkOpacityPercent = normalized?.opacityPercent
            ?: it.photoFrameWatermarkOpacityPercent,
        photoFrameWatermarkEffect = normalized?.effect
            ?: it.photoFrameWatermarkEffect,
    )
}
```

```
}
/**
 * 将 Photo Picker 返回的临时 URI 复制为内容寻址的私有文件。调用方只持有摘要，原图随后
 * 被移动、删除或系统回收 URI 权限，都不会影响预览和导出。
 */
fun importPhotoFrameWatermarkImage(
    sourceUri: Uri,
    onComplete: (Result<String>) -> Unit,
) {
    if (!LicenseManager.isPro.value) {
        onComplete(Result.failure(IllegalStateException("Custom watermark requires Pro")))
        return
    }
    viewModelScope.launch {
        val result = withContext(Dispatchers.IO) {
            storePhotoFrameWatermarkImage(
                context = getApplication(),
                resolver = contentResolver,
                sourceUri = sourceUri,
            )
        }
        onComplete(result)
    }
}

/** 保存监看页的应用内横屏状态；不改变 Android Activity 的系统方向。 */
fun setRemoteRotation(rotation: Int) {
    prefs.edit().putInt("remote_rotation", rotation).apply()
    _state.update { it.copy(remoteRotation = rotation) }
}

/** 应用内语言；写入后需 Activity.recreate() 才对界面生效（attachBaseContext 重读偏好）。 */
fun setAppLanguage(tag: String) {
    prefs.edit().putString(AppLocale.PREF_KEY, tag).apply()
    _state.update { it.copy(appLanguage = tag) }
}

/** 应用筛选（类型/保护/连拍/未传输，面板点击后即时提交）。持久化。 */
fun setFilters(
    exts: Set<String>?,
    protectedOnly: Boolean,
    burstOnly: Boolean,
    untransferredOnly: Boolean
) {
    prefs.edit().apply {
        if (exts == null) remove("filter_exts") else putStringSet("filter_exts", exts)
        if (protectedOnly) putBoolean("filter_protected", true) else remove("filter_protected")
        if (burstOnly) putBoolean("filter_burst", true) else remove("filter_burst")
        if (untransferredOnly) putBoolean("filter_untransferred", true) else remove("filter_untransferred")
    }.apply()
    _state.update {
        it.copy(
            filterExtensions = exts,

```



```

        filterProtectedOnly = protectedOnly,
        filterBurstOnly = burstOnly,
        filterUntransferredOnly = untransferredOnly
    )
}
}
fun setTransferDirUri(uri: Uri) {
    contentResolver.takePersistableUriPermission(
        uri,
        android.content.Intent.FLAG_GRANT_READ_URI_PERMISSION or
        android.content.Intent.FLAG_GRANT_WRITE_URI_PERMISSION
    )
    prefs.edit().putString("transfer_dir", uri.toString()).apply()
    _state.update { it.copy(transferDirUri = uri.toString(), existingExportFiles = emptyMap()) }
    viewModelScope.launch(Dispatchers.IO) { refreshExistingExportFiles(uri, deleteParts = false) }
}
private fun refreshExistingExportFiles(uri: Uri, deleteParts: Boolean) {
    try {
        val existing = sweepAndListExisting(uri, deleteParts).sizes
        val normalized = HashMap<String, MutableSet<Long>>()
        existing.forEach { (name, size) ->
            // 派生图只参与边框输出的文件名防冲突，不参与“相机原片已传”状态。
            // 否则每传一张 JPG 都会让 Compose 持有的索引多一项，目录越大越浪费。
            if (
                !isPhotoFrameOutputName(name) &&
                !name.equals(PHOTO_FRAME_OUTPUT_DIRECTORY, ignoreCase = true)
            ) {
                normalized.getOrPut(baseName(name)) { HashSet() }.add(size)
            }
        }
        _state.update { state ->
            if (state.transferDirUri == uri.toString()) {
                // 扫描期间可能已有新传输完成并写入索引；合并而不是覆盖，避免慢扫描
                // 用启动时的旧目录快照抹掉刚完成文件的绿勾。
                state.existingExportFiles.forEach { (name, sizes) ->
                    normalized.getOrPut(name) { HashSet() }.addAll(sizes)
                }
                state.copy(existingExportFiles = normalized.mapValues { it.value.toSet() })
            } else state
        }
    } catch (_: Exception) {
        // 保留扫描期间由已完成传输写入的索引；新目录初始本来就是空映射。
    }
}
private fun recordExistingExport(uri: Uri, name: String, size: Long) {
    val normalizedName = baseName(name)
    _state.update { state ->
        // 目录选择器在传输期间仍可能被打开；旧目录任务完成后绝不能污染新目录索引。
        if (state.transferDirUri != uri.toString()) return@update state
        val sizes = state.existingExportFiles[normalizedName].orEmpty() + size
    }
}

```

```

        state.copy(existingExportFiles = state.existingExportFiles + (normalizedName to sizes))
    }
}

fun addToQueue(files: List<NikonCamera.FileInfo>, cameraProvider: () -> NikonCamera?) {
    val snapshot = _state.value
    val dirUri = snapshot.transferDirUri ?: return
    val newTasks = createQueueTasks(
        files = files,
        photoFrameEnabled = snapshot.photoFrameEnabled,
        photoFrameBorderEnabled = snapshot.photoFrameBorderEnabled,
        photoFramePreset = snapshot.photoFramePreset,
        photoFrameWatermark = snapshot.photoFrameWatermark,
    )
    if (newTasks.isEmpty()) return
    _state.update { state -> state.copy(tasks = state.tasks + newTasks) }
    processQueue(dirUri, cameraProvider)
}

private fun processQueue(dirUri: String, cameraProvider: () -> NikonCamera?) {
    if (transferJob?.isActive == true) return
    transferJob = viewModelScope.launch {
        val self = coroutineContext[Job]
        _state.update { it.copy(isTransferring = true) }
        var serviceStarted = false
        try {
            val uri = Uri.parse(dirUri)
            val docUri = DocumentsContract.buildDocumentUriUsingTree(
                uri,
                DocumentsContract.getTreeDocumentId(uri)
            )
            // 队列启动前先校验传输目录仍然存在且可访问：目录被删除/改名/换存储后，
            // 后续 createDocument 会抛 "Missing file for primary:..." 这类系统原始
            // 错误直接漏到界面上。失效则清掉设置——用户下次点图会被既有引导
            // （未设目录自动弹设置面板）带去重新选择。
            val dirValid = withContext(Dispatchers.IO) {
                try {
                    contentResolver.query(
                        docUri,
                        arrayOf(DocumentsContract.Document.COLUMN_DOCUMENT_ID),
                        null, null, null
                   )?.use { true } ?: false
                } catch (_: Exception) {
                    false
                }
            }
            if (!dirValid) {
                prefs.edit().remove("transfer_dir").apply()
                // 文案只取一次：str() 每次都要构建配置上下文，放 map 里会按任务数重复执行
                // （update 遇 CAS 重试还会整体重跑）。
                val dirInvalidMsg = str(R.string.error_dir_invalid)
                _state.update { s ->

```

```

        s.copy(
            transferDirUri = null,
            tasks = s.tasks.map {
                if (it.status == TransferStatus.WAITING) {
                    it.copy(status = TransferStatus.FAILED, error = dirInvalidMsg)
                } else it
            }
        )
    }
    return@launch // finally 负责复位 isTransferring (前台服务尚未启动)
}
// 单次遍历：保留半成品(.nkpart_)供断点续传 + 建立“已存在(名称->大小)”去重表。
val directoryScan = withContext(Dispatchers.IO) {
    sweepAndListExisting(uri, deleteParts = false)
}
val existing = directoryScan.sizes
val existingUris = directoryScan.uris
val partFiles = directoryScan.parts
while (true) {
    // taskId 标识队列任务；handle 只用于与相机通信。同一 handle 可以有多张
    // 不同装饰配置的任务卡，任何状态更新都不能再按 handle 批量命中。
    val task = _state.value.tasks.firstOrNull { it.status == TransferStatus.WAITING } ?: break
    val taskId = task.taskId
    val handle = task.file.handle
    // 第一查：原片存在就直接引用，不再区分“下载任务/边框任务”。
    val localOriginal = findLocalOriginal(
        file = task.file,
        existingSizes = existing,
        existingUris = existingUris,
    )
    if (localOriginal != null) {
        log { "DL_SKIP existing: ${task.file.fileName}" }
        recordExistingExport(uri, localOriginal.displayName, localOriginal.size)
        val preset = task.framePreset
        if (preset == null) {
            updateTask(taskId) {
                it.copy(
                    status = TransferStatus.COMPLETED,
                    skipped = true,
                    progress = 1f,
                    downloaded = localOriginal.size,
                    speed = 0,
                )
            }
        }
    } else {
        // 第二查必须发生在启动前台服务之前。若派生图已经存在，任务会瞬间
        // 完成；此时先 startForegroundService 再立即 stop，会在部分系统上
        // 触发 ForegroundServiceDidNotStartInTimeException，直接杀掉 App。
        val effectiveWatermark = effectivePhotoFrameWatermark(
            isPro = LicenseManager.isPro.value,

```

```
        preference = task.frameWatermarkRequested,
        borderEnabled = task.frameBorderRequested,
    )
    val destinationKey = uri.toString()
    val frameExists = withContext(photoFrameDispatcher) {
        val destination = photoFrameDestinations[destinationKey]
        ?: PhotoFrameExporter.prepareDestination(
            resolver = contentResolver,
            treeUri = uri,
        ).let { prepared ->
            photoFrameDestinations.putIfAbsent(
                destinationKey,
                prepared,
            ) ?: prepared
        }
        destination.hasFrameFor(
            localOriginal.displayName,
            preset,
            effectiveWatermark,
            borderEnabled = task.frameBorderRequested,
        )
    }
    if (frameExists) {
        log {
            "DERIVATIVE_SKIP existing: ${localOriginal.displayName} " +
            "border=${task.frameBorderRequested} preset=${preset.name}"
        }
        updateTask(taskId) {
            it.copy(
                status = TransferStatus.COMPLETED,
                skipped = true,
                progress = 1f,
                downloaded = localOriginal.size,
                speed = 0,
            )
        }
        continue
    }
    updateTask(taskId) {
        it.copy(
            status = TransferStatus.COMPLETED,
            progress = 1f,
            downloaded = localOriginal.size,
            speed = 0,
            isGeneratingFrame = true,
        )
    }
    if (!serviceStarted) {
        TransferService.start(getApplication(), useWifi = false)
        serviceStarted = true
    }
}
```

```

    }
    launchPhotoFrameExport(
        taskId = taskId,
        treeUri = uri,
        sourceUri = localOriginal.uri,
        sourceName = localOriginal.displayName,
        preset = preset,
        borderEnabled = task.frameBorderRequested,
        watermarkRequested = task.frameWatermarkRequested,
        skipIfExists = true,
        failTaskOnError = true,
    )
}
continue
}
// 免费版每日完成数已达上限：不再开始新传输，卡片直接标注并引导解锁。
// 检查放在“已存在跳过”之后——跳过不占额度，到了上限也照常放行。
if (LicenseManager.transferLimitReached()) {
    updateTask(taskId) {
        it.copy(
            status = TransferStatus.FAILED,
            error = str(R.string.transfer_limit_reached),
            speed = 0
        )
    }
    continue
}
// 免费版单文件超限（>400MB，长视频/Raw 连拍段）：同样入队不拦、
// 轮到才检，卡片标注引导解锁后跳过，队列继续传后面的。
// >4GB 对象的 size 是哨兵值，数值上必然超限，一并拦住。
if (LicenseManager.freeSizeLimitExceeded(task.file.size)) {
    updateTask(taskId) {
        it.copy(
            status = TransferStatus.FAILED,
            error = str(
                R.string.transfer_size_limit,
                LicenseManager.FREE_MAX_FILE_BYTES / (1024 * 1024)
            ),
            speed = 0
        )
    }
    continue
}
// 每个任务开始时现取相机实例：中途掉线重连后，后续任务用的是新连接，
// 而不是队列启动时捕获的旧实例（旧实例 socket 已死，只会全部快速失败）。
val camera = cameraProvider()
if (camera == null) {
    updateTask(taskId) {
        it.copy(status = TransferStatus.FAILED, error = str(R.string.camera_not_connected), speed
= 0)
    }
}

```

```

        continue
    }
    updateTask(taskId) { it.copy(status = TransferStatus.TRANSFERING) }
    log { "DL BEGIN: ${task.file.fileName} handle=$handle size=${task.file.size}" }
    // 首个真正要下载的文件才拉起前台服务（全部命中“已存在”时不必启动，避免通知闪一下）。
    if (!serviceStarted) {
        TransferService.start(
            getApplication(),
            useWifi = camera.connectionType == CameraConnectionType.WIFI
        )
        serviceStarted = true
    }
    // 断点续传：检查是否存在上次传输留下的、【身份令牌匹配】的半成品文件。
    var resumeOffset = 0L
    var fileDocUri: Uri? = null
    val partFile = partFiles[task.file.fileName]?.takeIf { it.token == identityToken(task.file) }
    if (partFile != null) {
        val partSize = partFile.size
        // task.file.size 对 >4GB 文件是 SIZE_UNKNOWN 哨兵，绝不能拿它当真实大小比较。
        val sizeKnown = task.file.size > 0 && task.file.size != PtpConstants.SIZE_UNKNOWN
        if (sizeKnown && partSize >= task.file.size) {
            // 半成品已达完整大小：上次下载完在改名前崩了，直接改名跳过下载。
            // 仅在大小【已知】时走此捷径——SIZE_UNKNOWN 下 partSize>=哨兵会把
            // 4.3GB 的截断视频误判为完整，造成静默数据丢失。
            log { "DL_RESUME_COMPLETE: ${task.file.fileName} partSize=$partSize" }
            val finalName = task.file.fileName
            var renamed = renameQuietly(partFile.uri, finalName)
            if (renamed == null) {
                // 改名失败：复用已有副本逻辑
                for (n in 1..99) {
                    val candidate = suffixedName(finalName, n)
                    if (existing.containsKey(candidate)) continue
                    renamed = renameQuietly(partFile.uri, candidate)
                    if (renamed != null) break
                }
            }
            if (renamed != null) {
                val savedName = displayNameOf(renamed) ?: finalName
                existing[savedName] = partSize
                existingUris[savedName] = renamed
                recordExistingExport(uri, savedName, partSize)
                // 回到循环顶部，统一走“原片存在 → 检查边框”的同一条路径。
                continue
            } else {
                // 改不了，删半成品让正常路径重下
                deleteQuietly(partFile.uri)
            }
        } else if (partSize >= RESUME_CHUNK_SIZE && (!sizeKnown || partSize < task.file.size)) {
            // 半成品够大（≥1 块）且未完整：从块边界续传。大小未知(>4GB)也允许——
            // 由协议层用 GetObjectSize 解析真实大小后做全文件完整性校验。

```

```

        resumeOffset = (partSize / RESUME_CHUNK_SIZE) * RESUME_CHUNK_SIZE
        fileDocUri = partFile.uri
        log { "DL_RESUME: ${task.file.fileName} partSize=$partSize resumeOffset=$resumeOffset" }
    } else {
        // 太小 (<64MB) 或异常半成品, 删掉重建。
        deleteQuietly(partFile.uri)
    }
}
try {
    // SAF 的建文件/开流/关闭冲刷都是跨进程 Binder + 磁盘 IO, 放 IO 线程,
    // 不在主线程随每个文件抖一下 (状态更新经 StateFlow.update, 线程安全)。
    val result = withContext(Dispatchers.IO) {
        if (fileDocUri == null) {
            // 新建临时文件
            val createdUri = DocumentsContract.createDocument(
                contentResolver,
                docUri,
                getMimeType(task.file.fileName),
                partFileName(task.file)
            ) ?: throw Exception(str(R.string.error_create_file))
            fileDocUri = createdUri
        }
        // 续传时用 ParcelFileDescriptor "rw" 模式实现 seekable 写入;
        // 新文件用 openOutputStream (截断写入, 行为不变)。
        val outputStream: java.io.OutputStream
        if (resumeOffset > 0) {
            val pfd = contentResolver.openFileDescriptor(fileDocUri!!, "rw")
                ?: throw Exception(str(R.string.error_open_file))
            // AutoCloseOutputStream 持有 pfd 所有权: BufferedOutputStream.use {} 关闭
            // 输出流时一并 close 掉 pfd, 既不泄漏 fd, 也确保 DocumentsProvider 收到
            // 写完成信号后才发生改名 (裸 FileOutputStream(pfd.fileDescriptor) 两者皆失)。
            val fos = ParcelFileDescriptor.AutoCloseOutputStream(pfd)
            fos.channel.position(resumeOffset)
            outputStream = fos
        } else {
            outputStream = contentResolver.openOutputStream(fileDocUri!!)
                ?: throw Exception(str(R.string.error_open_file))
        }
        // 用大缓冲包裹 SAF 输出流, 把零散的写批量化, 减少 ContentProvider 往返。
        // 缺了它, 每个 PTP-IP 数据包都要跨 Binder 写一次 SAF, 吞吐直接腰斩 (2M/s→<1M/s)。
        java.io.BufferedOutputStream(outputStream, 1024 * 1024).use { out ->
            camera.downloadToFile(
                handle, out,
                onProgress = { progress ->
                    val speed = if (progress.elapsed > 0) {
                        (progress.downloaded / progress.elapsed).toLong()
                    } else 0
                    _state.update { state ->
                        state.copy(
                            currentSpeed = speed,

```

```

        tasks = state.tasks.map { t ->
            if (t.taskId == taskId && t.status == TransferStatus.TRANSFERING)
{
                t.copy(
                    progress = if (progress.total > 0) {
                        progress.downloaded.toFloat() / progress.total
                    } else 0f,
                    downloaded = progress.downloaded,
                    speed = speed
                )
            } else t
        }
    )
}
},
resumeOffset = resumeOffset,
totalSize = task.file.size
)
}
}
// withContext 正常返回则 fileDocUri 必已赋值。
val createdUri = checkNotNull(fileDocUri)
result.fold(
    onSuccess = { stats ->
        // 下载完整 → 把临时名改成真正文件名（相机上报的文件名即为准）。
        val finalName = task.file.fileName
        var savedName = finalName
        var renamedUri = if (renameBroken) null else renameQuietly(createdUri, finalName)
        if (renamedUri == null && !renameBroken) {
            for (n in 1..99) {
                val candidate = suffixedName(finalName, n)
                if (existing.containsKey(candidate)) continue
                renamedUri = renameQuietly(createdUri, candidate)
                if (renamedUri != null) {
                    savedName = candidate
                    break
                }
            }
        }
        var saveError: Throwable? = null
        if (renamedUri == null) {
            var copyName = finalName
            if (existing.containsKey(copyName)) {
                for (n in 1..99) {
                    val candidate = suffixedName(finalName, n)
                    if (!existing.containsKey(candidate)) {
                        copyName = candidate
                        break
                    }
                }
            }
        }
    }
}

```



```
        val copied = copyAsFallback(
            docUri, createdUri, copyName,
            getMimeType(finalName), stats.bytes
        )
        val copiedUri = copied.getOrNull()
        if (copiedUri != null) {
            renameBroken = true
            deleteQuietly(createdUri)
            savedName = displayNameOf(copiedUri) ?: copyName
            renamedUri = copiedUri
            log { "DL_SAVE via copy fallback: $savedName (rename broken)" }
        } else {
            saveError = copied.exceptionOrNull()
        }
    }
}

if (renamedUri != null) {
    existing[savedName] = stats.bytes
    existingUris[savedName] = renamedUri
    recordExistingExport(uri, savedName, stats.bytes)
    val framePreset = task.framePreset
    val shouldGenerateFrame = framePreset != null
    // 起点由协议层在取得相机 IO 独占权后记录；这里仍是正式文件
    // 已落盘并完成改名/复制后的完成点。
    val elapsed = android.os.SystemClock.elapsedRealtime() -
        stats.startedAtElapsedMs
    // 免费额度按“真正传输完成”计数(此处是唯一完成点；
    // 跳过/续传改名捷径都不经过这里，不计)。
    LicenseManager.recordTransferDone()
    updateTask(taskId) {
        it.copy(
            status = TransferStatus.COMPLETED, progress = 1f,
            downloaded = stats.bytes, speed = 0,
            downloadMbps = stats.mbps,
            elapsedMs = elapsed,
            isGeneratingFrame = shouldGenerateFrame,
        )
    }
}

if (framePreset != null) {
    // 派生严格发生在正式原片落盘之后。导出器只读取原片并创建
    // 新文件；独立低优先级线程立即接管，传输循环直接处理下一张。
    // 无论解码/写入是否失败，都不回滚、不删除原片。
    launchPhotoFrameExport(
        taskId = taskId,
        treeUri = uri,
        sourceUri = renamedUri,
        sourceName = savedName,
        preset = framePreset,
        borderEnabled = task.frameBorderRequested,
        watermarkRequested = task.frameWatermarkRequested,
        skipIfExists = true,
    )
}
```

```

        )
    }
} else {
    // 改名与复制均失败：删掉临时文件并标记失败——
    // 重试时从头下载（改名失败不是传输层问题，续传解决不了）。
    deleteQuietly(createdUri)
    val reason = when {
        saveError is java.io.FileNotFoundException ->
            str(R.string.error_dir_invalid)
        saveError?.message != null -> saveError.message
        else -> str(R.string.error_rename_copy_refused)
    }
    updateTask(taskId) {
        it.copy(status = TransferStatus.FAILED, error = str(R.string.error_save_faile
d, reason), speed = 0)
    }
}
},
onFailure = { e ->
    if (e is ResumeUnavailableException) {
        // 走不了续传（相机不支持分块 / >4GB 拿不到真实大小）：删掉半成品，
        // 本次标记失败，重试将从头全新下载——绝不用错位的全量数据续写。
        deleteQuietly(fileDocUri)
        updateTask(taskId) {
            it.copy(status = TransferStatus.FAILED, error = str(R.string.transfer_faile
d), speed = 0)
        }
    } else {
        // 普通传输失败：保留半成品，重试时从块边界续传。
        // 不删 .nkp*_：断点续传依赖它，交给 App 启动 init 的 sweep 统一清扫。
        updateTask(taskId) {
            it.copy(status = TransferStatus.FAILED, error = friendlyError(e), speed = 0)
        }
    }
}
}
} catch (e: CancellationException) {
    // 协程取消只发生在 ViewModel 销毁（App 退出）时——界面上的“停止”
    // 不再取消协程（正在传的文件自然传完，见 withdrawPending）。
    // 此处【不】就地删除半成品：半成品交给 App 启动的 sweep 统一清扫
    // (.nkp*_ 前缀带前导点，相册中本就不可见），也是断点续传的基础。
    throw e
} catch (e: Exception) {
    // 异常保留半成品——不是传输层错误（如目录失效），但半成品仍有价值
    // 保留可让用户修好设置后重试时续传。
    if (BuildConfig.DEBUG) {
        android.util.Log.e(TAG, "DL_FAIL: ${task.file.fileName} - ${e.javaClass.simpleName}: ${e.
message}", e)
    }
    updateTask(taskId) {
        it.copy(status = TransferStatus.FAILED, error = friendlyError(e), speed = 0)
    }
}
}
}

```

```

    }
    } catch (cancelled: CancellationException) {
        throw cancelled
    } catch (error: Exception) {
        // 目录 URI 解析、目录扫描等位于单任务 try/catch 之外。任何 provider
        // 异常都只能让相关卡片失败，不能成为主线程未捕获异常导致 App 闪退。
        if (BuildConfig.DEBUG) {
            android.util.Log.e(
                TAG,
                "QUEUE_FAIL: ${error.javaClass.simpleName}: ${error.message}",
                error,
            )
        }
    }
    val message = friendlyError(error)
    _state.update { state ->
        state.copy(tasks = state.tasks.map { task ->
            if (
                task.status == TransferStatus.WAITING ||
                task.status == TransferStatus.TRANSFERING
            ) {
                task.copy(
                    status = TransferStatus.FAILED,
                    error = message,
                    speed = 0,
                )
            } else {
                task
            }
        })
    }
    } finally {
        // 仅当本协程仍是当前传输 job 时才收尾，避免误停新队列的前台服务/误清传输
        // 状态（旧队列收尾期间新队列可能已启动并接管 transferJob）。
        if (transferJob === self) {
            _state.update { it.copy(isTransferring = false, currentSpeed = 0) }
            stopTransferServiceIfIdle()
        }
    }
}

/**
 * 单次遍历目标目录：
 * 1) 当 [deleteParts]=true 时删除遗留的半成品（[PART_PREFIX] 开头的临时文件，上次崩溃/被杀留下）；
 * 同时删除旧进程遗留的边框临时文件；当前进程会话的边框任务始终保留；
 * 2) 返回完整文件的 显示名->大小/Uri，用于“已存在则跳过”及已传原片的本地派生；
 * 3) 收集半成品文件信息到 parts 映射（原文件名 -> PartInfo），用于断点续传。
 * 合并清扫与列举，避免两次全目录扫描；正常完成的文件已改真名，不会被误删。
 *
 * @param deleteParts true=清空半成品（App 启动/新队列），false=保留半成品供续传（队列启动重试）
 * @return 完整文件大小、Uri 与半成品映射的一致快照

```

```

*/
private fun sweepAndListExisting(
    treeUri: Uri,
    deleteParts: Boolean = true
): ExistingDirectoryScan {
    val map = HashMap<String, Long>()
    val uris = HashMap<String, Uri>()
    val parts = HashMap<String, PartInfo>()
    try {
        val childrenUri = DocumentsContract.buildChildDocumentsUriUsingTree(
            treeUri,
            DocumentsContract.getTreeDocumentId(treeUri)
        )
        contentResolver.query(
            childrenUri,
            arrayOf(
                DocumentsContract.Document.COLUMN_DISPLAY_NAME,
                DocumentsContract.Document.COLUMN_SIZE,
                DocumentsContract.Document.COLUMN_DOCUMENT_ID
            ),
            null, null, null
       )?.use { c ->
            val nameIdx = c.getColumnIndex(DocumentsContract.Document.COLUMN_DISPLAY_NAME)
            val sizeIdx = c.getColumnIndex(DocumentsContract.Document.COLUMN_SIZE)
            val idIdx = c.getColumnIndex(DocumentsContract.Document.COLUMN_DOCUMENT_ID)
            if (nameIdx >= 0) {
                while (c.moveToNext()) {
                    val name = c.getString(nameIdx) ?: continue
                    if (name.startsWith(PHOTO_FRAME_PART_PREFIX)) {
                        // 边框派生临时文件不可续传: App 启动时清理; 队列运行期间
                        // 只忽略不删除, 避免新队列扫描误删仍在后台写入的旧队列任务。
                        if (
                            deleteParts &&
                            !isCurrentPhotoFrameTempName(name) &&
                            idIdx >= 0
                        ) {
                            val docId = c.getString(idIdx) ?: continue
                            runCatching {
                                DocumentsContract.deleteDocument(
                                    contentResolver,
                                    DocumentsContract.buildDocumentUriUsingTree(treeUri, docId),
                                )
                            }
                        }
                    }
                }
            } else if (name.startsWith(PART_PREFIX)) {
                if (deleteParts && idIdx >= 0) {
                    val docId = c.getString(idIdx) ?: continue
                    try {
                        DocumentsContract.deleteDocument(
                            contentResolver,

```

```

        DocumentsContract.buildDocumentUriUsingTree(treeUri, docId)
    )
    } catch (_: Exception) {}
} else if (!deleteParts && idIdx >= 0) {
    // 续传模式：保留半成品，解析出身份令牌与原文件名（按首个下划线切分）。
    val docId = c.getString(idIdx) ?: continue
    val size = if (sizeIdx >= 0 && !c.isNull(sizeIdx)) c.getLong(sizeIdx) else 0L
    val afterPrefix = name.removePrefix(PART_PREFIX)
    val sep = afterPrefix.indexOf('_')
    if (sep > 0) {
        val token = afterPrefix.substring(0, sep)
        val origName = afterPrefix.substring(sep + 1)
        if (origName.isNotEmpty()) {
            parts[origName] = PartInfo(
                uri = DocumentsContract.buildDocumentUriUsingTree(treeUri, docId),
                size = size, token = token
            )
        }
    }
}
// sep<=0: 旧格式/异常半成品名，不记录（App 启动 init sweep 会清掉）。
}
} else {
    val size = if (sizeIdx >= 0 && !c.isNull(sizeIdx)) c.getLong(sizeIdx) else -1L
    map[name] = size
    if (idIdx >= 0) {
        val docId = c.getString(idIdx)
        if (docId != null) {
            uris[name] = DocumentsContract.buildDocumentUriUsingTree(
                treeUri,
                docId,
            )
        }
    }
}
}
}
}
} catch (_: Exception) {}
return ExistingDirectoryScan(sizes = map, uris = uris, parts = parts)
}
/**
 * 删除下载失败/取消留下的半成品文件，忽略删除失败。走 IO 线程（Binder + 磁盘）；
 * NonCancellable 保证取消路径（协程已被 cancel）也能完成清理。
 */
private suspend fun deleteQuietly(uri: Uri?) {
    if (uri == null) return
    withContext(NonCancellable + Dispatchers.IO) {
        try {
            DocumentsContract.deleteDocument(contentResolver, uri)
        } catch (_: Exception) {}
    }
}

```

```

    }
}
/** 改名：失败（如目标名已存在、部分 provider 返回 null）返回 null。走 IO 线程。 */
private suspend fun renameQuietly(uri: Uri, newName: String): Uri? = withContext(Dispatchers.IO) {
    try {
        DocumentsContract.renameDocument(contentResolver, uri, newName)
    } catch (_: Exception) {
        null
    }
}
/**
 * 鸿蒙回退：把下载完好的临时文件【复制】成名为 [name] 的正式文件（renameDocument
 * 损坏的设备用，见 [renameBroken]）。复制后按 [expectedBytes] 校验字节数，
 * 不完整/失败则删除半成品正式文件并返回 null。
 * 权衡说明：改名是原子操作，复制不是——进程在复制中途被杀会留下残缺真名文件
 * （本地磁盘拷贝很快，窗口极小，且残缺文件因大小不符不会被“已存在跳过”误放行，
 * 重传会以 “(n)” 副本落盘自愈）。仅在改名损坏的设备上承担此取舍。
 * 成功返回正式文件 Uri；临时文件的删除由调用方负责。走 IO 线程。
 */
private suspend fun copyAsFallback(
    parentDocUri: Uri,
    tempUri: Uri,
    name: String,
    mime: String,
    expectedBytes: Long
): Result<Uri> = withContext(Dispatchers.IO) {
    var created: Uri? = null
    try {
        created = DocumentsContract.createDocument(contentResolver, parentDocUri, mime, name)
        ?: return@withContext Result.failure(Exception(str(R.string.error_create_file)))
        val copiedBytes = contentResolver.openInputStream(tempUri)!!.use { input ->
            java.io.BufferedOutputStream(
                contentResolver.openOutputStream(created)!!, 1024 * 1024
            ).use { output ->
                input.copyTo(output, 1024 * 1024)
            }
        }
        if (copiedBytes != expectedBytes) throw Exception(str(R.string.error_copy_incomplete, copiedBytes, expectedBytes))
        Result.success(created)
    } catch (e: CancellationException) {
        // 取消（App 退出）不吞：清掉半成品后向上传播，维持“取消必须传播”的全局约定。
        created?.let {
            try { DocumentsContract.deleteDocument(contentResolver, it) } catch (_: Exception) {}
        }
        throw e
    } catch (e: Exception) {
        if (BuildConfig.DEBUG) {
            android.util.Log.w(TAG, "copyAsFallback failed: ${e.message}")
        }
        created?.let {

```

```

        try { DocumentsContract.deleteDocument(contentResolver, it) } catch (_: Exception) {}
    }
    // 带原因返回：界面把它拼进“保存失败：…”，出错自带诊断信息，免大范围排查。
    Result.failure(e)
}
}

/** 查询 SAF 文档的实际显示名（provider 重名时会静默改名，落盘名≠请求名）。走 IO 线程。 */
private suspend fun displayNameOf(uri: Uri): String? = withContext(Dispatchers.IO) {
    try {
        contentResolver.query(
            uri, arrayOf(DocumentsContract.Document.COLUMN_DISPLAY_NAME), null, null, null
       )?.use { c -> if (c.moveToFirst()) c.getString(0) else null }
    } catch (_: Exception) {
        null
    }
}

/** 剥掉重名副本后缀：“DSC_0001 (2).NEF” -> “DSC_0001.NEF”，用于与相机文件名归一化匹配。 */
private fun baseName(name: String): String = name.replace(COPY_SUFFIX_REGEX, "")
/** 从本次目录快照中找到已传原片；优先精确文件名，其次接受同大小的重名副本。 */
private fun findLocalOriginal(
    file: NikonCamera.FileInfo,
    existingSizes: Map<String, Long>,
    existingUris: Map<String, Uri>,
): LocalOriginal? {
    fun sizeMatches(localSize: Long): Boolean =
        localSize < 0L ||
        file.size == PtpConstants.SIZE_UNKNOWN ||
        localSize == file.size
    val exactSize = existingSizes[file.fileName]
    val exactUri = existingUris[file.fileName]
    if (exactSize != null && exactUri != null && sizeMatches(exactSize)) {
        return LocalOriginal(file.fileName, exactSize, exactUri)
    }
    return existingSizes.entries.firstNotNullOfOrNull { (displayName, size) ->
        val localUri = existingUris[displayName] ?: return@firstNotNullOfOrNull null
        if (baseName(displayName).equals(file.fileName, ignoreCase = true) && sizeMatches(size)) {
            LocalOriginal(displayName, size, localUri)
        } else {
            null
        }
    }
}

/** 生成重名副本名：“DSC_0001.NEF” + 2 -> “DSC_0001 (2).NEF”；无扩展名则直接追加。 */
private fun suffixedName(name: String, n: Int): String {
    val dot = name.lastIndexOf('.')
    return if (dot <= 0) "$name ($n)" else "${name.substring(0, dot)} ($n)${name.substring(dot)}"
}

/**
 * 把原片派生移出相机传输协程。单线程调度器保证位图内存峰值可控，低线程优先级让
 * 相机 IO 优先；任务数单独计数，使最后一张边框完成前前台服务不会被提前停止。
 */

```

```
*/
private fun launchPhotoFrameExport(
    taskId: Long,
    treeUri: Uri,
    sourceUri: Uri,
    sourceName: String,
    preset: PhotoFramePreset,
    borderEnabled: Boolean,
    watermarkRequested: PhotoFrameWatermark,
    skipIfExists: Boolean = false,
    failTaskOnError: Boolean = false,
) {
    activePhotoFrameExports.incrementAndGet()
    val job = viewModelScope.launch(photoFrameDispatcher) {
        val destinationKey = treeUri.toString()
        val outcome = try {
            val destination = photoFrameDestinations[destinationKey]
                ?.PhotoFrameExporter.prepareDestination(
                    resolver = contentResolver,
                    treeUri = treeUri,
                ).let { prepared ->
                    photoFrameDestinations.putIfAbsent(destinationKey, prepared) ?: prepared
                }
            // 在真正轮到渲染时重新判定授权：排队期间若高级版到期，免费版默认
            // 水印立即恢复；查重与导出必须使用同一份生效配置。
            val effectiveWatermark = effectivePhotoFrameWatermark(
                isPro = LicenseManager.isPro.value,
                preference = watermarkRequested,
                borderEnabled = borderEnabled,
            )
            if (
                skipIfExists &&
                destination.hasFrameFor(
                    sourceName,
                    preset,
                    effectiveWatermark,
                    borderEnabled = borderEnabled,
                )
            ) {
                FrameExportOutcome.AlreadyExists
            } else {
                PhotoFrameExporter.export(
                    context = getApplication(),
                    resolver = contentResolver,
                    destination = destination,
                    sourceUri = sourceUri,
                    sourceName = sourceName,
                    preset = preset,
                    watermark = effectiveWatermark,
                    borderEnabled = borderEnabled,
                )
            }
        } catch (e: Exception) {
            failTaskOnError && Log.e("Z相机", "导出失败: ${e.message}")
        }
    }
```



```

        ).fold(
            onSuccess = { FrameExportOutcome.Saved(it.displayName) },
            onFailure = { FrameExportOutcome.Failed(it) },
        )
    }
} catch (cancelled: CancellationException) {
    throw cancelled
} catch (error: Exception) {
    FrameExportOutcome.Failed(error)
}
}
when (outcome) {
    is FrameExportOutcome.Saved -> {
        log { "DERIVATIVE_SAVE: $sourceName -> ${outcome.displayName}" }
    }
    FrameExportOutcome.AlreadyExists -> {
        log {
            "DERIVATIVE_SKIP existing: $sourceName " +
            "border=${borderEnabled} preset=${preset.name}"
        }
        updateTask(taskId) { task ->
            task.copy(
                status = TransferStatus.COMPLETED,
                skipped = true,
                error = null,
            )
        }
    }
}
is FrameExportOutcome.Failed -> {
    // 子目录若被用户在运行期间删除，下一个任务重新查找/创建。
    photoFrameDestinations.remove(destinationKey)
    log {
        "DERIVATIVE_FAILED: $sourceName " +
        "${outcome.error.javaClass.simpleName}: ${outcome.error.message}"
    }
    if (failTaskOnError) {
        updateTask(taskId) { task ->
            task.copy(
                status = TransferStatus.FAILED,
                error = friendlyError(outcome.error),
                speed = 0,
            )
        }
    }
}
}
}
}
// invokeOnCompletion 即使任务排队期间就被取消也必定执行，计数和 UI 不会泄漏。
job.invokeOnCompletion {
    updateTask(taskId) { task ->
        if (task.isGeneratingFrame) task.copy(isGeneratingFrame = false) else task
    }
}

```

```

    }
    if (activePhotoFrameExports.decrementAndGet() == 0) {
        stopTransferServiceIfIdle()
    }
}

private fun stopTransferServiceIfIdle() {
    if (!_state.value.isTransferring && activePhotoFrameExports.get() == 0) {
        TransferService.stop(getApplication())
    }
}

/** 按 taskId 就地更新单个任务；ID 不存在时保持列表不变。用 update 保证跨线程原子读改写。 */
private fun updateTask(taskId: Long, transform: (TransferTask) -> TransferTask) {
    _state.update { state ->
        state.copy(tasks = state.tasks.map { if (it.taskId == taskId) transform(it) else it })
    }
}

private inline fun log(message: () -> String) {
    if (BuildConfig.DEBUG) android.util.Log.d(TAG, message())
}

private fun getMimeType(fileName: String): String {
    return when {
        fileName.endsWith(".jpg", true) || fileName.endsWith(".jpeg", true) -> "image/jpeg"
        fileName.endsWith(".png", true) -> "image/png"
        fileName.endsWith(".nef", true) -> "image/x-nikon-nef"
        fileName.endsWith(".mov", true) -> "video/quicktime"
        fileName.endsWith(".mp4", true) -> "video/mp4"
        else -> "application/octet-stream"
    }
}

/**
 * 撤下所有等待中的任务（WAITING→CANCELLED），队列协程不会再开始它们；
 * 正在传输的文件让它自然传完——中途打断需要发 PTP/IP Cancel 包或直接断开连接，
 * 实测两者都会让相机挂起会话甚至关闭 Wi-Fi，代价远高于传完当前文件。
 * 队列协程发现没有 WAITING 任务后自然收尾（isTransferring 复位、前台服务停止）。
 * “清空队列”的第一步：先撤下，UI 播完移除动画后再逐个 [removeTask]。
 */
fun withdrawPending() {
    _state.update { state ->
        state.copy(
            tasks = state.tasks.map {
                if (it.status == TransferStatus.WAITING) {
                    it.copy(status = TransferStatus.CANCELLED, speed = 0)
                } else it
            }
        )
    }
}

/** 撤下单个等待中的任务（仅 WAITING→CANCELLED）：移除动画播放期间队列不得开始传它。 */
fun withdrawTask(taskId: Long) {

```

```

        _state.update { state ->
            state.copy(
                tasks = state.tasks.map {
                    if (it.taskId == taskId && it.status == TransferStatus.WAITING) {
                        it.copy(status = TransferStatus.CANCELLED, speed = 0)
                    } else it
                }
            )
        }
    }
}

/**
 * 清空收尾：一次性移除所有已终结的任务（CANCELLED/COMPLETED/FAILED）。
 * “清空队列”的兜底——LazyColumn 只组合可见卡片，屏幕外的卡没有条目协程替它做
 * “动画后移除”，由本方法在可见卡片收合动画播完后统一清掉。
 * 与 [removeTask] 同规则：TRANSFERING/WAITING/正在生成派生图的一律保留。
 */
fun removeCleared() {
    _state.update { state ->
        state.copy(
            tasks = state.tasks.filter {
                it.status == TransferStatus.TRANSFERING ||
                it.status == TransferStatus.WAITING ||
                it.isGeneratingFrame
            }
        )
    }
}

/**
 * 把任务卡片从队列移除（移除动画结束后调用），返回是否真的移除了。
 * 拒绝移除 TRANSFERING（正在传输）与 WAITING（动画期间被“重试”重置回等待，
 * 说明用户想要它了），以及正在生成派生图的任务——竞态下调用方把卡片弹回原高即可。
 * 合法移除路径上任务必为 CANCELLED/COMPLETED/FAILED（等待中的在标记时已 withdraw）。
 */
fun removeTask(taskId: Long): Boolean {
    var removed = false
    _state.update { state ->
        val kept = state.tasks.filterNot {
            it.taskId == taskId &&
            it.status != TransferStatus.TRANSFERING &&
            it.status != TransferStatus.WAITING &&
            !it.isGeneratingFrame
        }
        removed = kept.size != state.tasks.size
        state.copy(tasks = kept)
    }
    return removed
}

/**
 * 重试失败/取消的任务：保留入队时锁定的边框与水印配置，只创建新的任务 ID
 * 并重置运行状态。这样用户切换设置后，重试仍然得到入队时选定的派生图。
 */

```

```
*/
fun retryFailed(cameraProvider: () -> NikonCamera?) {
    val snapshot = _state.value
    val dirUri = snapshot.transferDirUri ?: return
    val retryIds = snapshot.tasks.filter {
        it.status == TransferStatus.FAILED || it.status == TransferStatus.CANCELLED
    }.mapTo(HashSet()) { it.taskId }
    if (retryIds.isEmpty()) return
    _state.update { state ->
        state.copy(tasks = state.tasks.map {
            if (
                it.taskId in retryIds &&
                (it.status == TransferStatus.FAILED || it.status == TransferStatus.CANCELLED)
            ) {
                it.newAttempt()
            } else {
                it
            }
        })
    }
    processQueue(dirUri, cameraProvider)
}

/** 同 [retryFailed]: 只重置指定任务, 绝不影响同一照片的其它边框任务。 */
fun retrySingleTask(taskId: Long, cameraProvider: () -> NikonCamera?) {
    val snapshot = _state.value
    val task = snapshot.tasks.firstOrNull { it.taskId == taskId } ?: return
    if (task.status != TransferStatus.FAILED && task.status != TransferStatus.CANCELLED) return
    val dirUri = snapshot.transferDirUri ?: return
    _state.update { state ->
        state.copy(tasks = state.tasks.map {
            if (
                it.taskId == taskId &&
                (it.status == TransferStatus.FAILED || it.status == TransferStatus.CANCELLED)
            ) {
                it.newAttempt()
            } else {
                it
            }
        })
    }
    processQueue(dirUri, cameraProvider)
}

override fun onCleared() {
    super.onCleared()
    transferJob?.cancel()
    photoFrameDispatcher.close()
    // 兜底停止前台服务, 防止 VM 销毁后通知残留。
    TransferService.stop(getApplication())
}
}
```